

数据驱动的规划方法

丁文超

深蓝学院





规划方法Review



传统Rule Based 规划：

A*、RRT、Lattice Planner ...

数据驱动的预测方法：

多模态传感器的编码方法（栅格化表示、向量化表示、基于Transformer的方法）

输出的表达形式（多模态轨迹回归、轨迹分类、目标点预测）

数据驱动的轨迹规划！😊





大纲



生成模型

模仿学习

强化学习

深蓝学院





Diffusion-ES



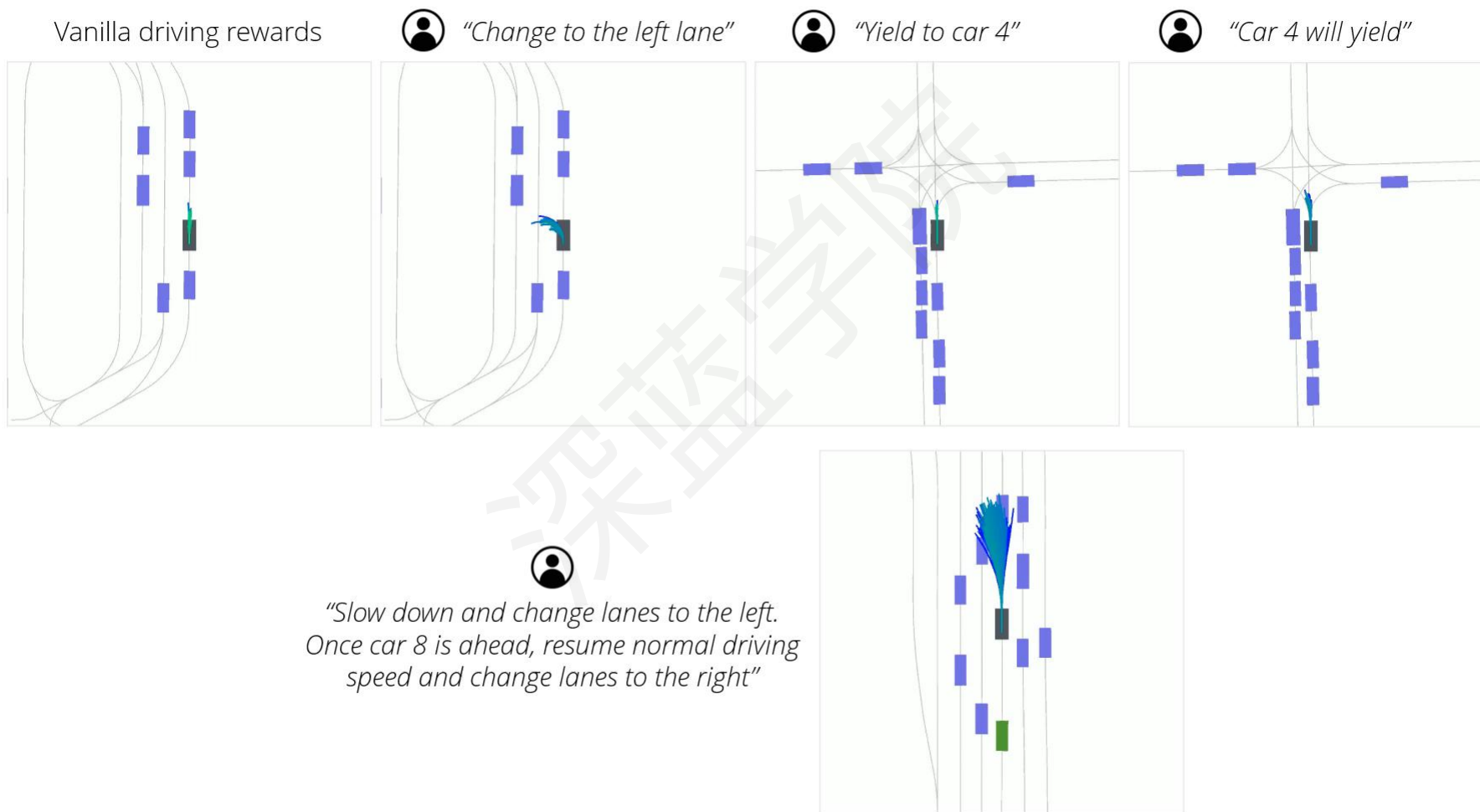
□ 背景:

- 传统优化方法复杂环境适应性较差;
- Diffusion model擅长对复杂多模态进行建模, 能够探索多种潜在的轨迹;
- Diffusion model的生成性质可以最大化黑箱奖励而无需梯度;
- Gradient-free Planning;
- 传统搜索过程效率低;
- 优化 few-shot LLM prompting reward functions(non-differentiable)





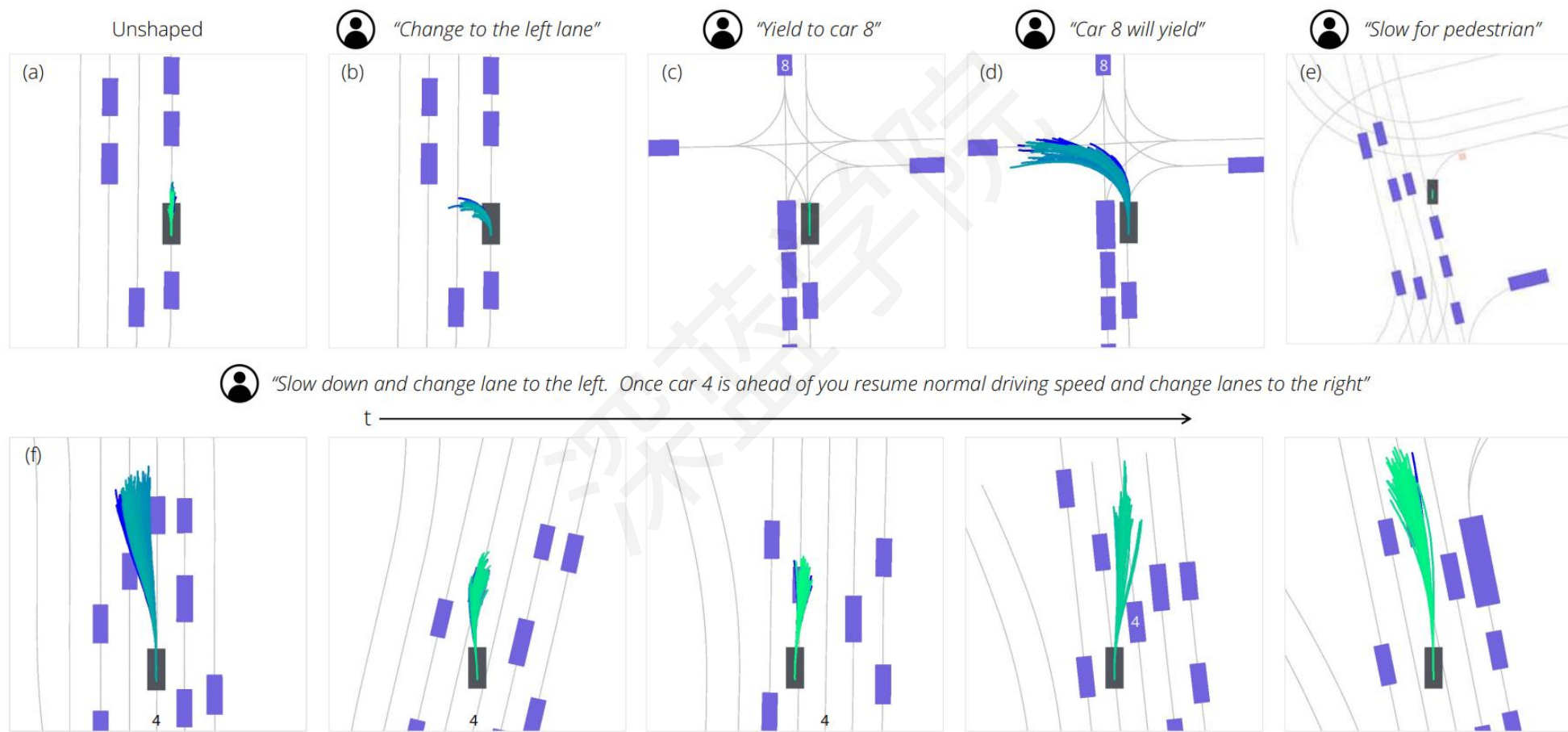
□ 视频演示:





Diffusion-ES: Gradient-free Planning with Diffusion for Autonomous Driving and Zero-Shot Instruction

Following



Diffusion-ES 工作演示图



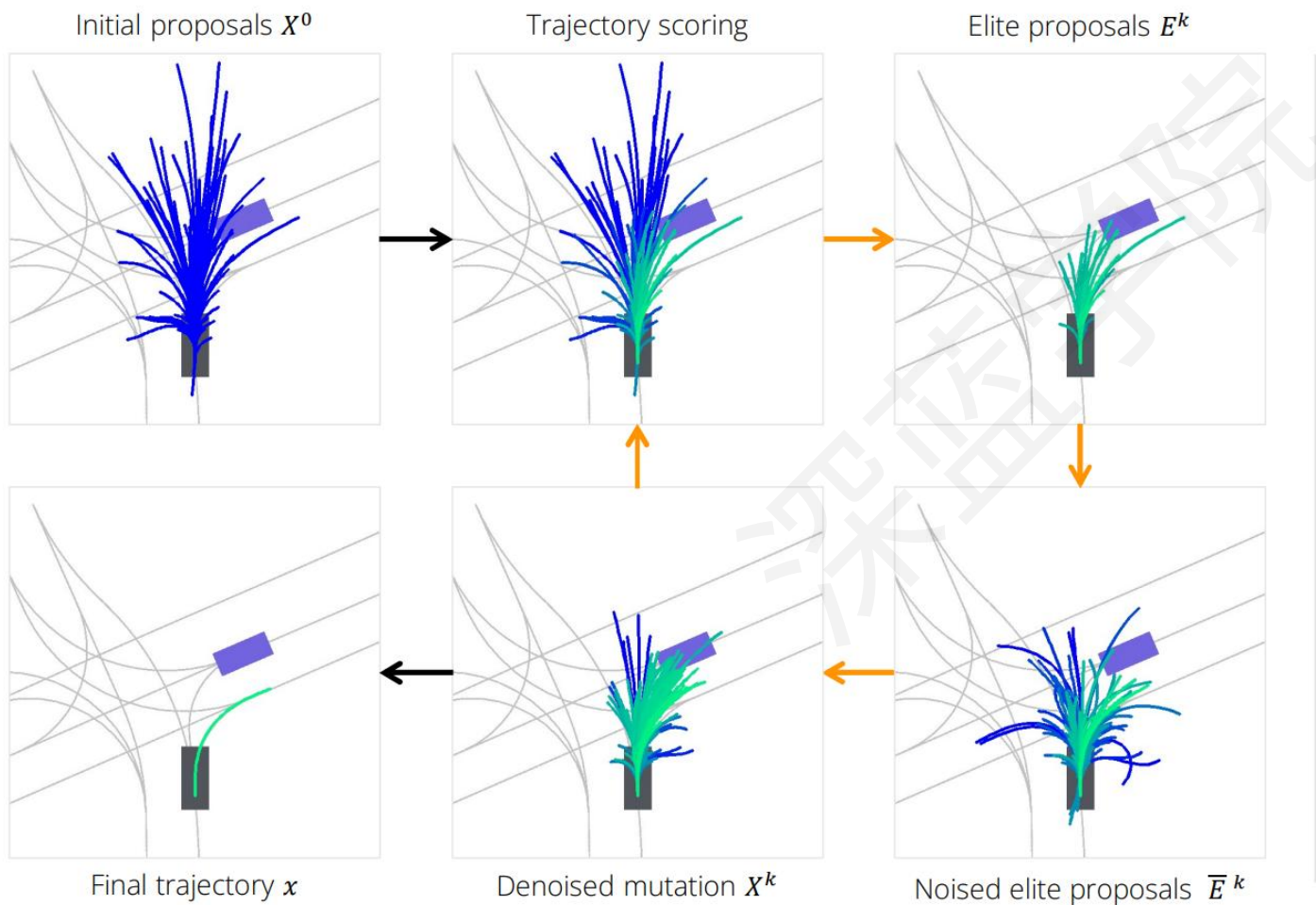


Diffusion-ES

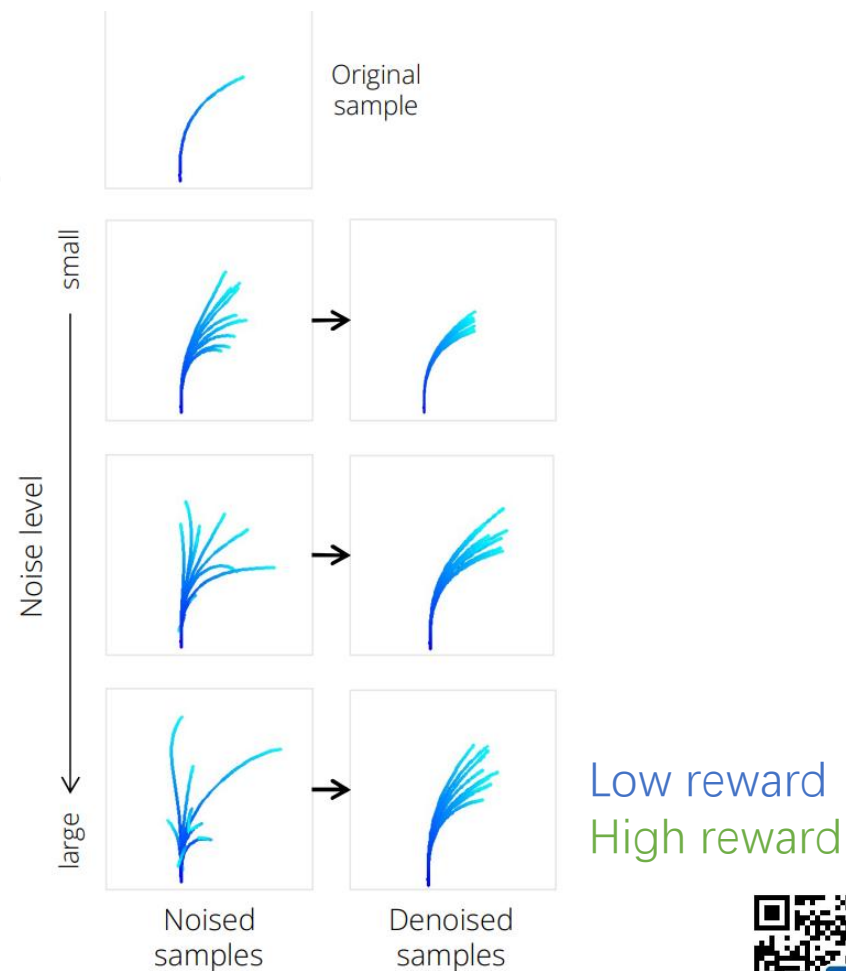


Diffusion-ES: Gradient-free Planning with Diffusion for Autonomous Driving and Zero-Shot Instruction

Following



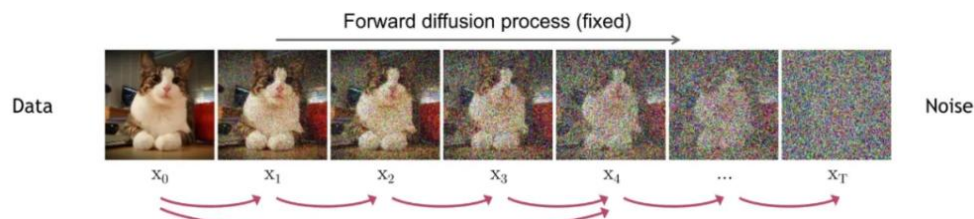
Diffusion-ES pipeline



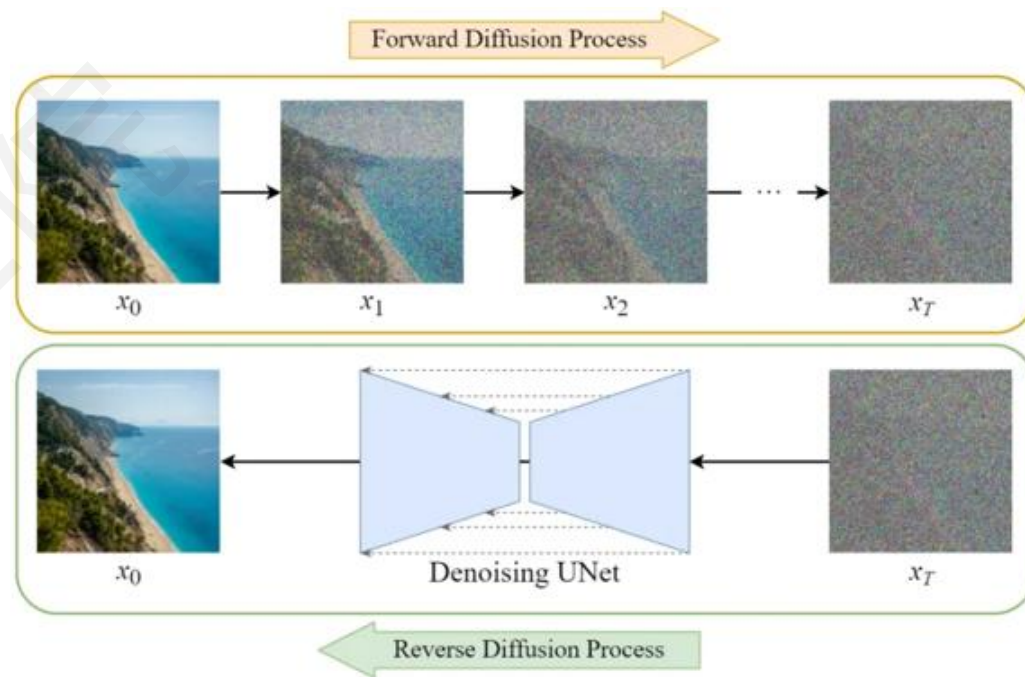
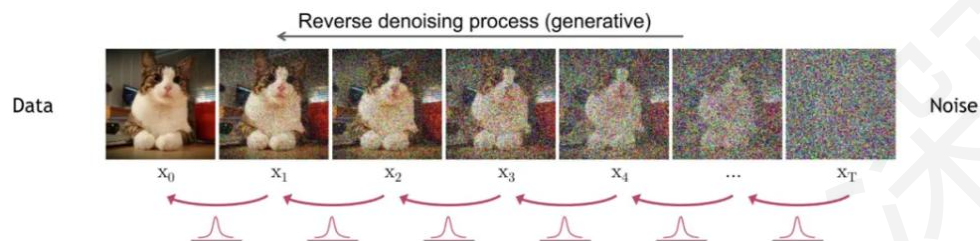


Diffusion model

- Diffusion model 通过前向扩散过程反转来获得概率分布，该过程逐渐将高斯噪声添加到初始样本
- Forward Diffusion Process: 添加噪声



- Reverse Diffusion Process: 去除噪声



Diffusion model

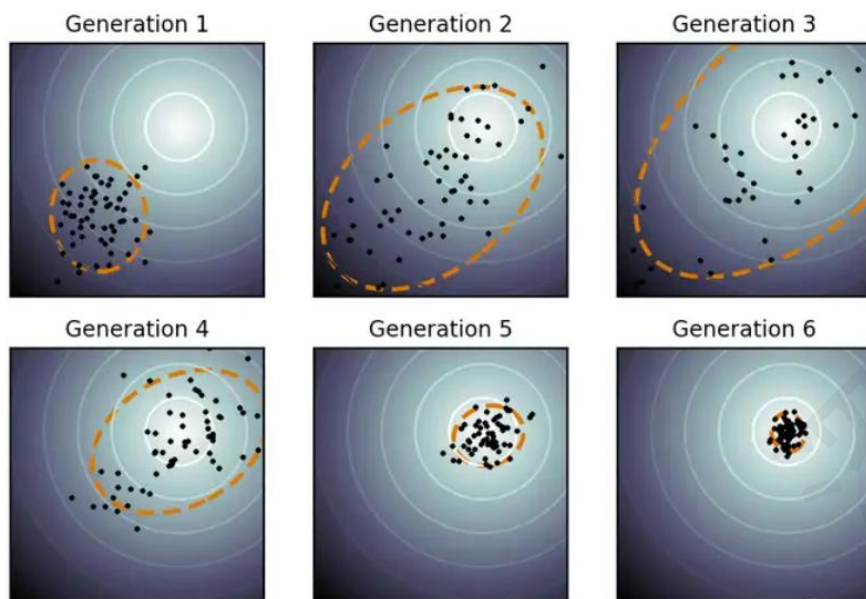
- 添加的噪声量取决于预定义的方差表 $\beta_t \in (0,1)_{t=1}^T$, T : 扩散时间步的总数
- 前向扩散: $x_t = \sqrt{\bar{\alpha}_t}x + \sqrt{1 - \bar{\alpha}_t}\epsilon$, $\epsilon \sim N(0,1)$ 与 x 具有相同维度, 其中 $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$
- 反向去噪: 神经网络 $\hat{\epsilon} = \epsilon_\theta(x_t; t)$, 输入噪声样本 x_t 、时间步长 t , 预测出添加的噪声 ϵ





□ Evolutionary Strategies

- 进化策略是一系列基于群体的无梯度优化算法，可以最大化任意黑盒奖励函数 $R(x): R^d \rightarrow R$ ，无需训练
- ES迭代更新搜索分布 $q(x)$ 以最大化预期奖励 $E_{x \sim p_\theta(x)}[R(x)]$



进化策略示意图





Diffusion-ES

Diffusion-ES Method

- 使用预训练好的扩散模型 ϵ_θ 初始化样本总体，并使用阶段的扩散去噪过程来变异样本
- 使用扩散模型对从M轨迹样本采样初始化总体 X^0

$$X^0 = \{x_i\}_{i=1}^M \sim p_\theta(x), \quad \text{其中 } X^k \text{ 是迭代 } k \text{ 次的总体样本}$$

- 样本评分：每迭代 k 次，都会对样本中的总体 $\{R(x_i) \mid x_i \in X^k\}_{i=1}^M$ 进行评分
- 样本选择：使用奖励决定哪些样本进行下一次迭代

$$q(x) = \frac{\exp(\tau R(x))}{\sum_{i=1}^M \exp(\tau R(x_i))}, \quad E^{k+1} = \{x_i\}_{i=1}^M \stackrel{\text{iid}}{\sim} q(x),$$

其中 E^{k+1} 表示精英集，在迭代 k 次中保留， τ 是控制 q 锐度的可调温度参数

- 样本突变：使用truncated Diffusion-denoising方法，随机对 E^{k+1} 进行突变。

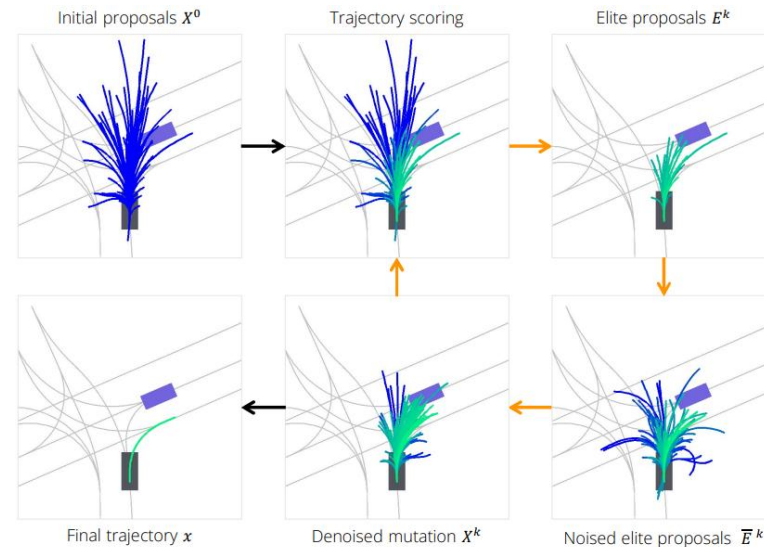
通过前向扩散过程前 t 步骤获得噪声精英样本 $\bar{E}^{k+1}$

$$E^{k+1} = \{\sqrt{\bar{\alpha}_N}x + \sqrt{1 - \bar{\alpha}_N}\epsilon \mid x \in E^{k+1}\}, \quad \text{其中 } \epsilon \sim N(0, 1)$$

- 然后运用反向扩散过程的最后 t 步，进行去噪从而得到样本 X^{k+1}

$$X^{k+1} = \{x \sim p_\theta(x \mid \bar{x}) \mid \bar{x} \in E^{k+1}\}, \quad \text{时间步长 } t \text{ 是一个超参数,}$$

控制每次迭代 k 的突变强度。



Algorithm 1 Diffusion-ES

- Input:** Diffusion model p_θ , reward function R , search steps K , population size M , number of noising steps N , variance schedule $\bar{\alpha} = \{\bar{\alpha}_i\}_{i=1}^N$.
- Initial proposals:** $X^0 \leftarrow \{x_i \sim p_\theta(x)\}_{i=1}^M$
- for** search step $k \in (1, \dots, K)$ **do**
- Score proposals $\{R(x_i) \mid x_i \in X^{k-1}\}_{i=1}^M$
- Compute distribution $q(x) = \frac{\exp(\tau R(x))}{\sum_{i=1}^M \exp(\tau R(x_i))}$
- Sample elites from X^{k-1} : $E^k \leftarrow \{x_i \stackrel{\text{iid}}{\sim} q(x)\}_{i=1}^M$
- Renoise elites $\bar{E}^k \leftarrow \{\sqrt{\bar{\alpha}_N}x + \sqrt{1 - \bar{\alpha}_N}\epsilon \mid x \in E^k, \epsilon \sim N(0, 1)\}$
- Denoise elites $X^k \leftarrow \{x \sim p_\theta(x \mid \bar{x}) \mid \bar{x} \in \bar{E}^k\}$
- end for**
- return** output $x = \arg \max_{x \in X^K} R(x)$





□ Mapping Language instructions to reward functions with LLM prompting

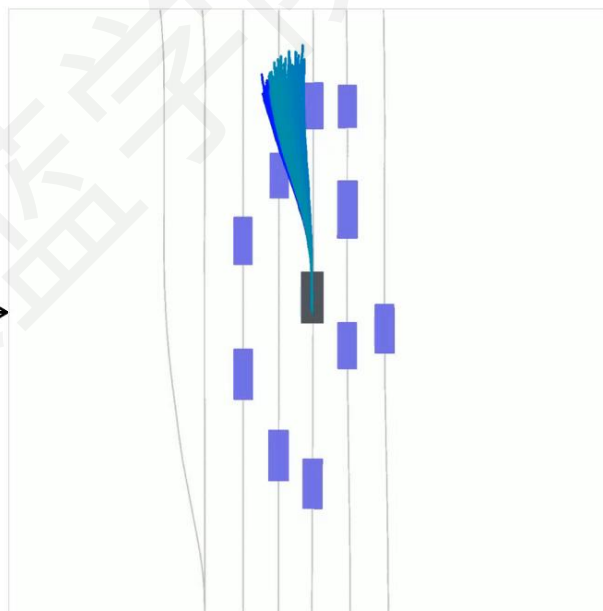
- 使用LLM从语言指令中合成奖励函数，其中奖励函数可以进行组合（可以通过语言指导与其他约束结合）



"Slow down and change lane to the left. Once car 4 is ahead of you resume normal driving speed and change lanes to the right"



```
current_speed = ego_vehicle.speed
add_ego_vel_reward(current_speed * 0.5)
lane = get_left_lane()
add_lane_follow_reward(lane)
while not reach_lane(lane):
    yield
def vehicle_ahead_of_us():
    vehicle = get_vehicle(4)
    return vehicle.is_ahead_of(ego_vehicle)
while not vehicle_ahead_of_us():
    yield
remove_ego_vel_reward()
lane = get_right_lane()
add_lane_follow_reward(lane)
while not reach_lane(lane):
    yield
```





□ Pros and Cons Discussion

- 通过Diffusion model解决轨迹生成，带给我们哪些新启发？
- Planning 中的non-differentiable问题 如何解决？
- Diffusion model的方法在闭环实验中的表现如何？
- Evolution strategies 的效率问题
- 从多模态的角度出发，Diffusion model的应用过程我们还可能再添加哪些模态的数据？

深蓝学院





GAD-Generative Learning for HD Map-Free Autonomous Driving



▣ 实车演示:



<https://www.youtube.com/watch?v=DYPB5ICRvn0>





GAD-Generative Learning for HD Map-Free Autonomous Driving



□ 背景:

- Corner case 不能通过列举手动制定的规则来解决
- 将预测、决策和规划模块结合来克服 rule-based难以解决的问题

深蓝学院

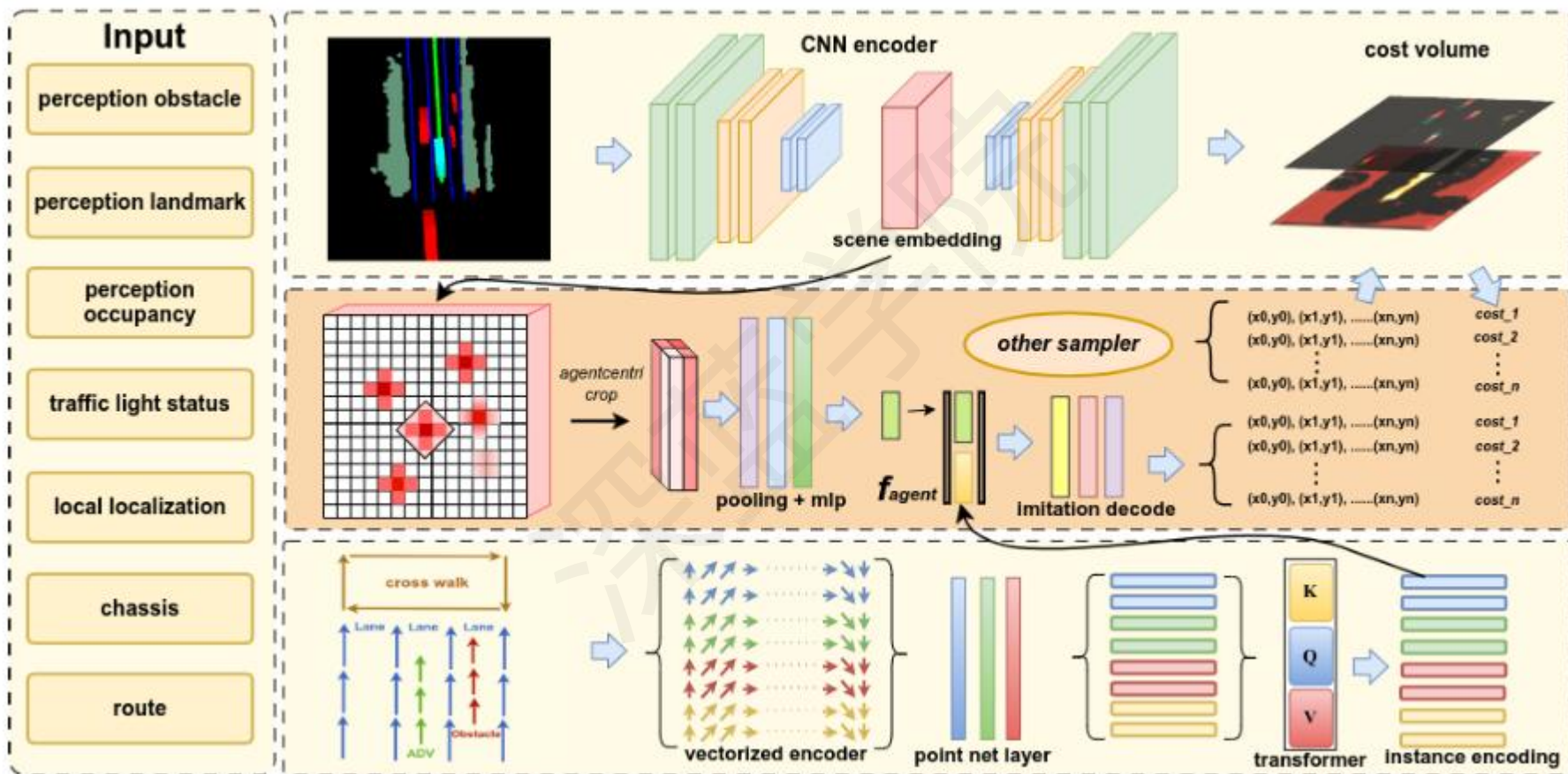




GAD-Generative Learning for HD Map-Free Autonomous Driving



Model Structure



GAD pipeline图



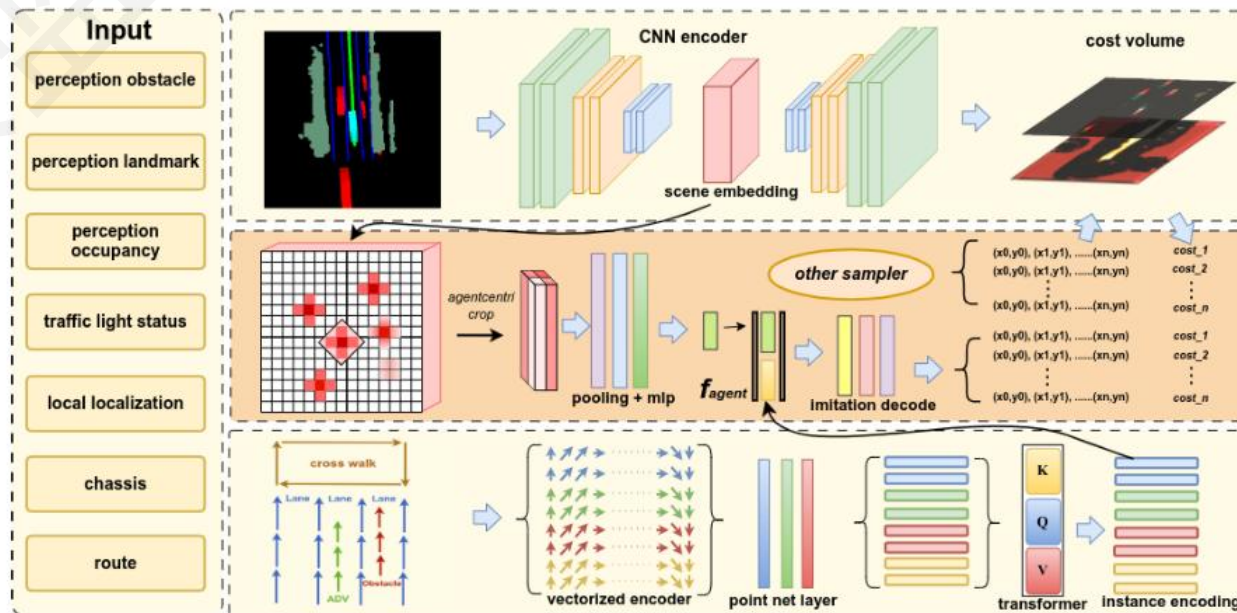


GAD-Generative Learning for HD Map-Free Autonomous Driving



Model Structure

- 光栅化stream使用多分辨率卷积编码器获得scene embedding;
- Scene embedding通过两个单独的反卷积解码器分别生成 cost volumes 和 predicted grid map;
- 处理矢量化图形流的encoding建立在层级式图网络结构的基础上:
 - PointNet-Based的局部子图, 用于处理来自矢量化输入的局部信息;
 - 使用Transformer编码器的全局图, 用于处理agents和landmark features之间的交互;
- 最后在以自车为中心的场特征中通过定向裁剪得到agent对应的场景编码;
- 将agent的场景编码与实例编码 (两个stream的embedding) 进行融合, 得到agent级别的特征 f_{agent} ;
- 使用模仿学习的decoder来为agent和ego产生轨迹。



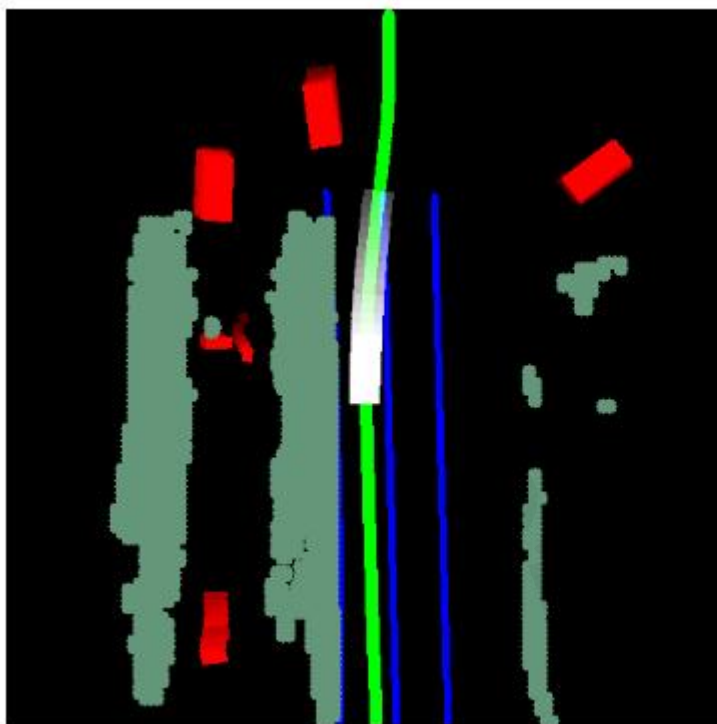


GAD-Generative Learning for HD Map-Free Autonomous Driving

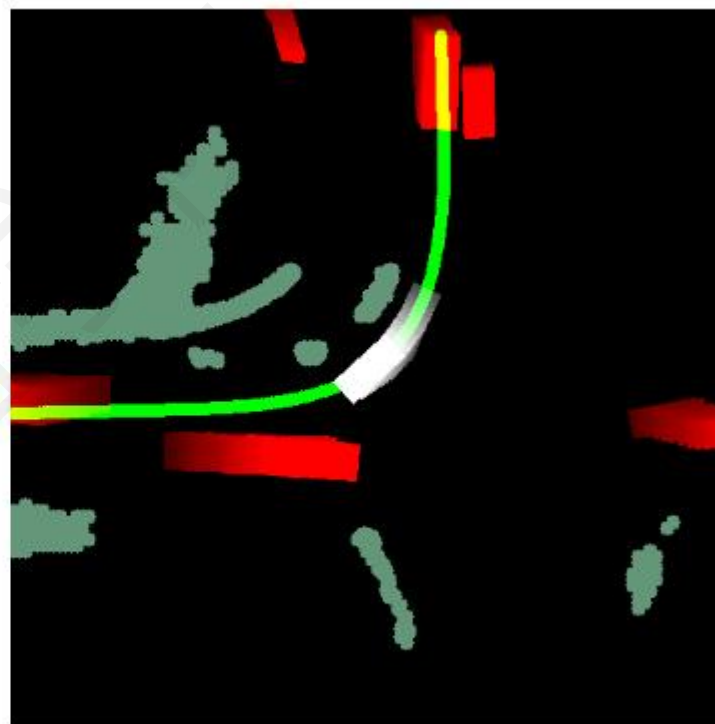


Route

- Route Information 来自于visual landmarks 和 human commuting trajectories



(a)



(b)

Route可视化图：a) 车道中心线被选作cruising scenarios的route；b) 历史通勤轨迹被选做turning scenarios的route, occupancy提供道路边界信息





GAD-Generative Learning for HD Map-Free Autonomous Driving



Multi-stage Prediction

- 第一阶段 **场景网格图**预测：通过解码场景的embedding，回归得到分辨率与栅格化输入匹配的占用栅格地图，其中Loss设计为：

$$L_{\text{pred_grid}} = - \sum_{ijt} [\tilde{g}_{ijt}(1 - \hat{g}_{ijt})^2 \log(\hat{g}_{ijt}) + (1 - \tilde{g}_{ijt})\hat{g}_{ijt}^2 \log(1 - \hat{g}_{ijt})]$$

$\tilde{g}_{ijt}$ 表示网格标签，在 t 时刻被占用式 $\tilde{g}_{ijt}=1$

- 第二阶段 **agent轨迹**预测：基于预测网格的场景先验，使用交互设计预测智能体轨迹。
- 通过累积计算网格概率得到附近landmarks的占用概率，过滤出高概率候选作为目标先验。

$$P_{\text{lane}} = \sum_t \left(\sum_{i,j \in (\text{lane})} g_{ij}^t \right)$$

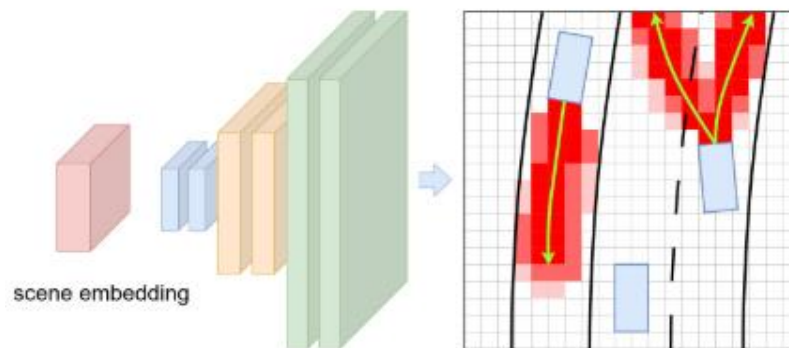
- 第三阶段 **轨迹**解码：基于agent 特征和车道编码来解码agent轨迹

$$\text{Traj} = \text{dec}(f_{\text{agent}} | f_{\text{lane}})$$

- 训练阶段：使用与agent未来轨迹对应的真实车道作为条件先验

$$L_{\text{traj_pred}} = L2(\text{Traj}, \text{dec}(f_{\text{agent}} | f_{\text{lane}}))$$

- 最终得到对于其他agent的**预测轨迹**



Multi-stage Prediction





GAD-Generative Learning for HD Map-Free Autonomous Driving



□ Trajectory Evaluation

- 通过时空cost volume，结合场景占用的预测网格图，从候选轨迹中选择**最优轨迹**
- 通过decoder 场景embedding 生成未来30帧的space-time cost volume

$$C_t^{xy} = \text{dec}(f_{\text{ras}})$$

- 轨迹代价为路径上点space-time cost之和，找到代价最小的轨迹

$$s = \arg \min_{s \in S} \sum_t [C(\hat{s}) + \alpha \text{Occ}(\hat{s}) + \beta \text{Grid}(\hat{s})]$$

其中， S 表示规划的轨迹集合，Occ为感知占用，Grid表示预测占用栅格， α 和 β 为超参数

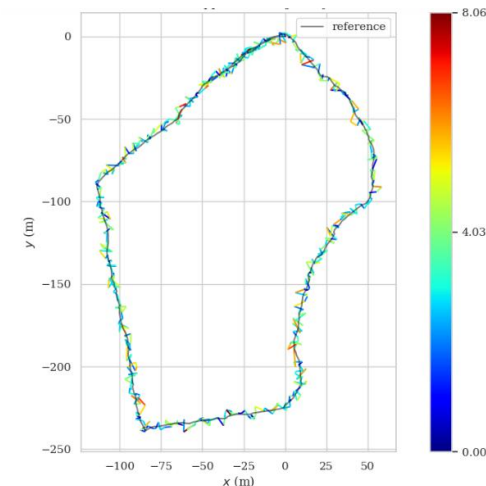
- 因为在离散集合中选择最小代价轨迹不可微，采用max-margin loss来惩罚偏离人类驾驶轨迹或不安全的轨迹，并鼓励人类轨迹 $\tilde{s}$ 的代价比其他轨迹 $\hat{s}$ 小：

$$L_{\text{cost_volume}} = \max_s [C(\tilde{s}) - C(\hat{s}) + \text{Diff}(\tilde{s}, \hat{s})]$$

其中， Diff 为差异度量，用于量化human demonstrations 和 algorithmic candidates之间的差距，评估规划目标、静态障碍物回避和动态交互调节等目标，eg:

$$\text{Diff}(\tilde{s}, \hat{s}) = L2(\tilde{s}, \hat{s}) + O(\hat{s}) + G_{rid}(\hat{s})$$

其中， O 表示occupancy cost， G_{rid} 表示其余的预测cost， $L2$ loss表示trajectory $\tilde{s}$ 与ground truth之间的距离。



Trajectory error map

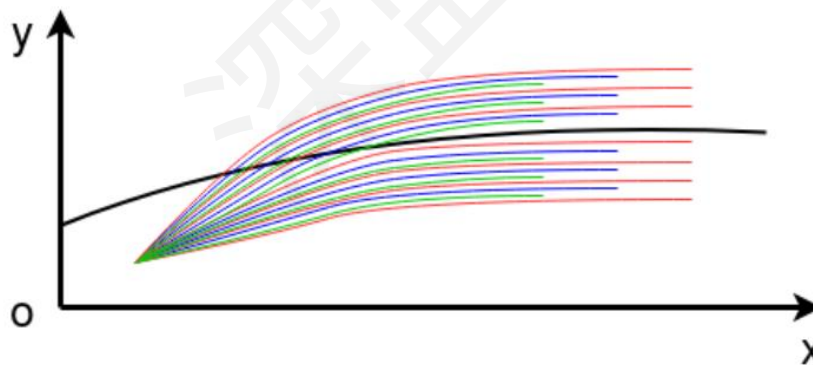




□ Trajectory Generation – Lattice Sampler

在Trajectory Evaluation中利用physical constraints优化得到最终轨迹是一个NP-hard问题，因此提出新的轨迹生成策略
轨迹采样和生成算法分为基于参考线和无参考线两类：

- 基于参考线的类型可以利用参考线信息保持长期一致的输出，增强跨帧稳定性，有助于轨迹的连续性；
- 无参考线的类型提高系统的可扩展性，适用于参考信息稀缺的场景。
- Lattice Sampler:
 - ① 车道中心线和周围智能体的运动都是潜在轨迹的强先验信息，如果存在landmark, lane geometry也可以用来指导轨迹的采样；
 - ② 可以结合预测的轨迹产生如跟随、超车等交互行为；
 - ③ 使用轨迹参数化和抽样化的方法，采样轨迹的动作（lane keeping、lane changing、nudging、yielding…）



Given the initial state $[s_0, s'_0, s''_0]$, $[l_0, l'_0, l''_0]$ at time t_0 , the end state $[s_1, s'_1, s''_1]$, $[l_1, l'_1, l''_1]$ at time t_1
Trajectories are sampled to cover cruising, following, overtaking, stopping maneuvers and so on.

Sampling in the Frenet frame, then converted back to the Cartesian coordinate system.





GAD-Generative Learning for HD Map-Free Autonomous Driving



□ Trajectory Generation – Imitation Planner

Imitation Planner: (mimic a human driver with one single trajectory? 😞)

- ① 估计多模态运动规划任务中的条件分布 $P(S|f_{plan})$, $P(S|f_{plan}) = \sum_M P(S|m, f_{plan})P(m|f_{plan})$

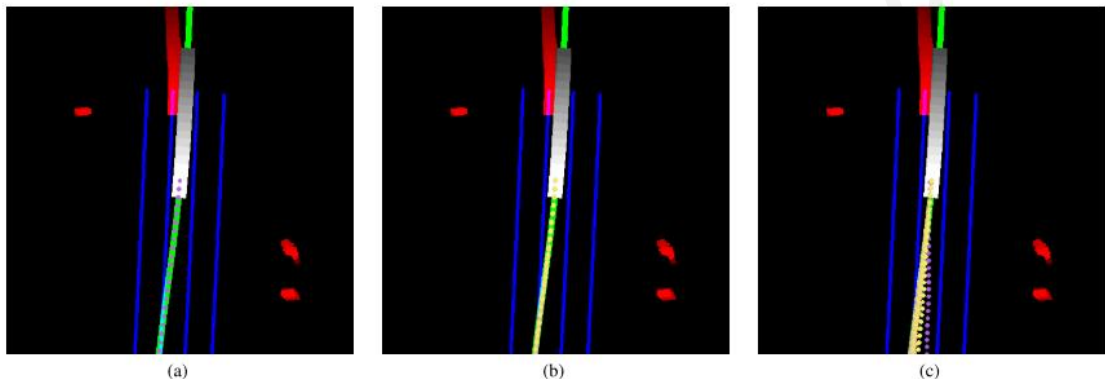
其中 $S = [\hat{s}_1, ..., \hat{s}_m]$ 是 m 中不同的策略, 如 lane keeping、agent following、agent overtaking, f_{plan} 表示规划任务的特征,

$f_{plan} = [f_{scene} \parallel f_{route} \parallel f_{ADV} \parallel f_{target_speed}]$, $\parallel$: concatenation

- ② Imitation generator 对 m 种可能的轨迹进行解码, 在训练过程中根据 $L2$ 距离识别出最接近 ground-truth 的轨迹 model

$$[\hat{s}_1, ..., \hat{s}_m] = G(f_{plan})$$

$$L_{imit_plan} = \arg \min_{i \in 1, ..., m} \text{dist}(\tilde{s}, \hat{s}_i)$$



Lane changing scenarios with multi trajectory generation

- a) Single trajectory
- b) Multi trajectories imitation
- c) GAN-based generation





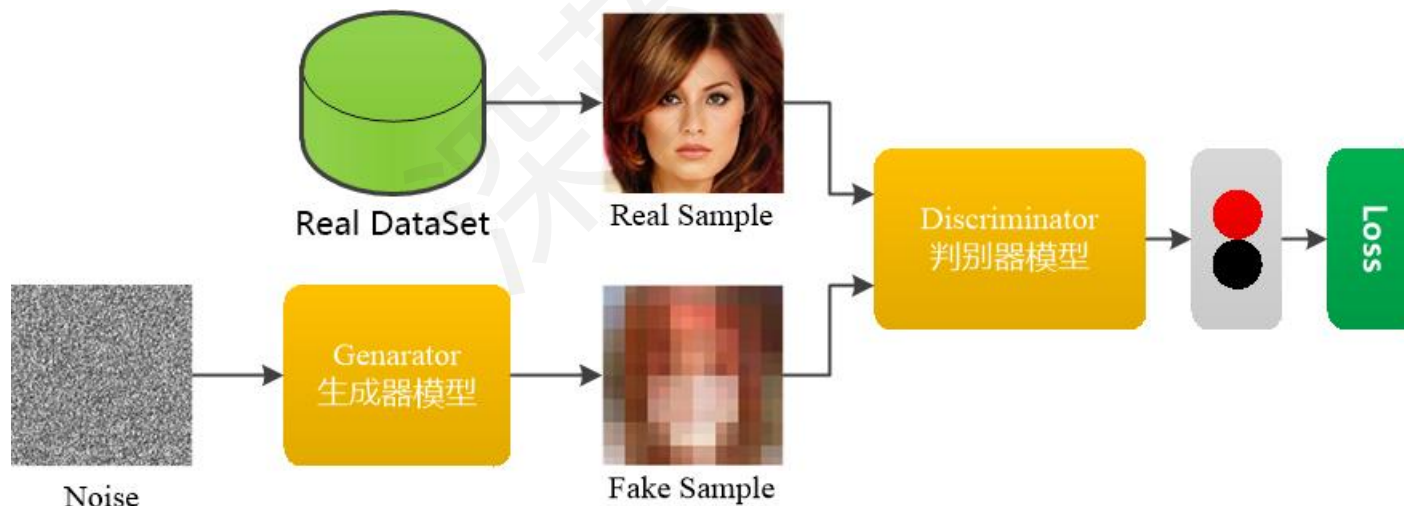
GAD-Generative Learning for HD Map-Free Autonomous Driving



□ Trajectory Generation – GAN-Based Planner

GAN-Based Planner:

- 基于GAN的规划器利用生成对抗网络（GAN）框架，通过学习生成多样化的轨迹，解决轨迹生成中的优化问题，提高规划器的鲁棒性。
- GAN框架:
 - ① 生成器（Generator）：生成逼真的轨迹，使其难以被判别器区分，学习从随机噪声输入到轨迹输出的映射；
 - ② 判别器（Discriminator）：区分真实轨迹和生成轨迹，通过与生成器对抗训练，逐步提升生成器的能力。





GAD-Generative Learning for HD Map-Free Autonomous Driving



□ Trajectory Generation - GAN-Based Planner

- 生成器G和判别器D的**对抗损失函数**为：

$$L_{\text{gan}} = \min_G \max_D [E_x[\log(D(x))] + E_z[\log(1 - D(G(z)))]]$$

- 生成器**的损失函数包括对抗损失和模仿损失两部分：

$$L_{\text{generator}} = L_{\text{gan}} + L_{\text{imit plan}}(\bar{s}, G(m))$$

- 评估器**（即GAN框架下的判别器）的损失函数为：

$$L_{\text{evaluator}} = \max_{\hat{s}} [E(\bar{s}) - E(\hat{s}) + \text{Diff}(\bar{s}, \hat{s})], \text{ 其中 } E(s) = \sum C(s) + \alpha O_{cc}(s) + \beta G_{rid}(s)$$

符号：

x 为真实轨迹样本；

z 为生成器的输入噪声；

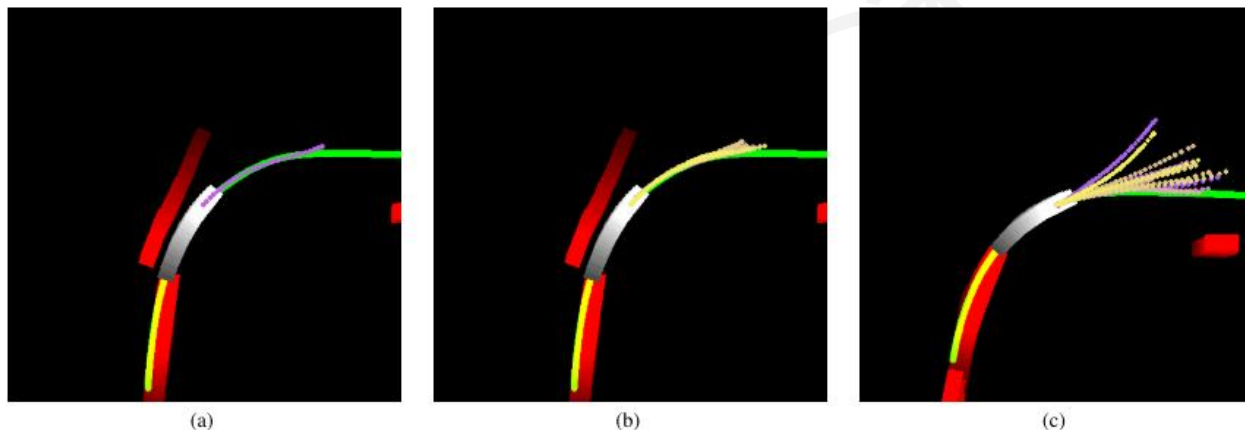
$G(z)$ 为生成轨迹；

$\log(D(x))$ 指判别器可以正确分类real case的概率，最大化 $\log(1 - D(G(z)))$ 可以帮助标记来自生成器的样本。

符号：

L_{gan} 表示生成器的对抗损失；

$L_{\text{imit plan}}(\bar{s}, G(m))$ 表示生成轨迹与真实轨迹的模仿损失。



Right turning scenarios with multi trajectory generation

- a) Single trajectory
- b) Multi trajectories imitation
- c) GAN-based generation





GAD-Generative Learning for HD Map-Free Autonomous Driving



□ Trajectory Generation - GAN-Based Planner

GAN-Based Planner:

- 生成器G根据输入的随机噪声m生成多样化的轨迹:

$$\hat{S} = [\hat{s}_1, \dots, \hat{s}_m] = G(m); m \sim \mathcal{N}(\mu, \sigma^2)$$

- 生成的轨迹 $\hat{S}$ 通过评估器以评估质量

深蓝学院





GAD-Generative Learning for HD Map-Free Autonomous Driving



□ Pros and Cons Discussion

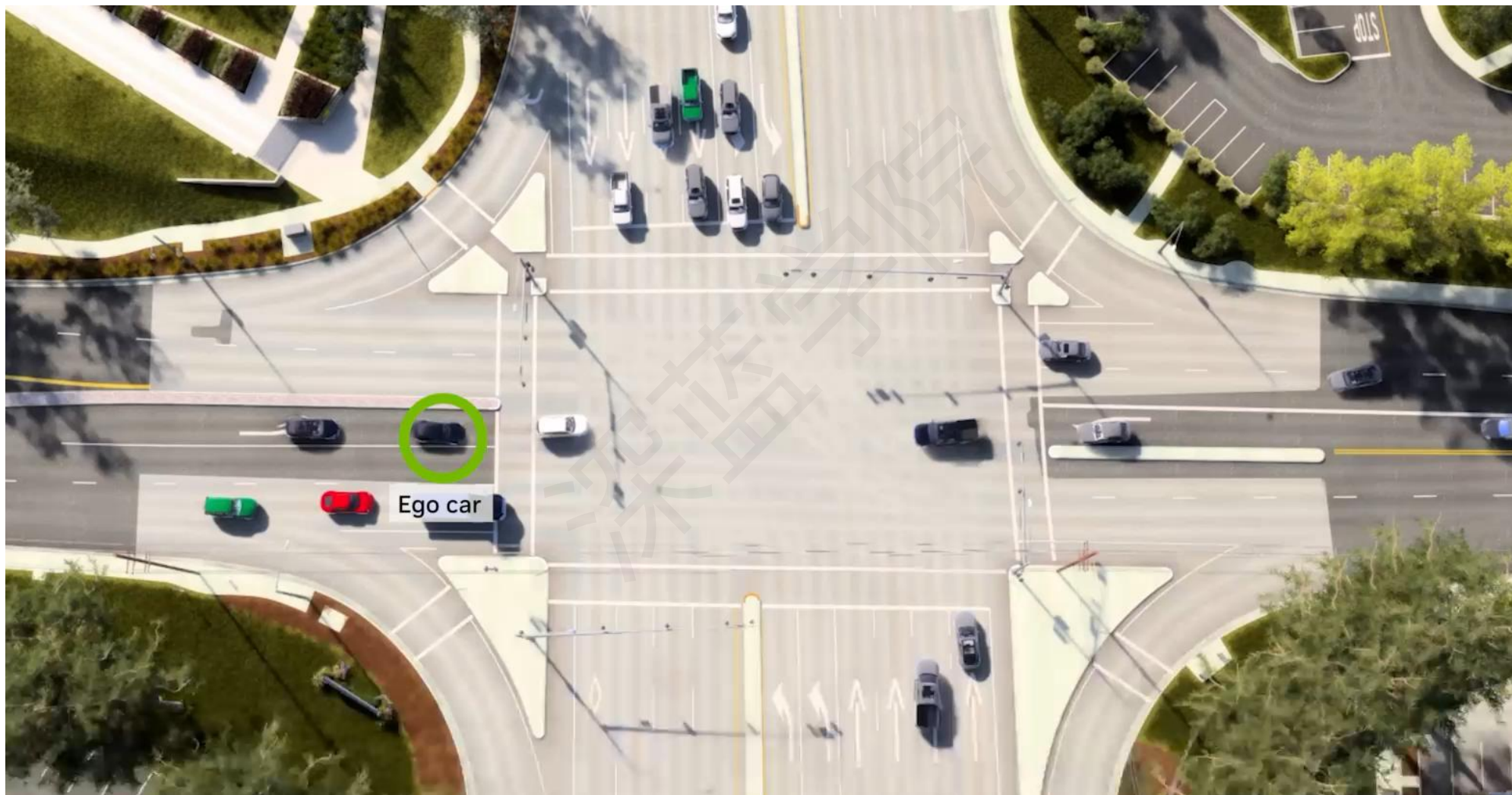
- 通过GAN架构得到candidate trajectories是一种全新的思路;
- 在Map-Free的前提下Route模块中是否还存在其他可以利用的信息, 如 Traffic Flow?
- 在Multi-stage Prediction层面的设计能否加入考虑自车action的影响?
- GAN的架构产生的类人轨迹可能存在什么问题, 如“模仿学习数据集单一模态轨迹不具有代表性”?

深蓝学院



UniGen驾驶场景生成

□ 生成具有挑战性的驾驶场景





□ 如何大规模生成稀有长尾场景

- 仿真环境中人工设计 -- 复杂性和多样性不足
- 利用深度学习模型生成场景，已有的做法

$$p(S|R) = \underbrace{p_\phi(S_0|R)} \underbrace{p_\psi(S_{1..T}|S_0, R)}$$

上述分开的建模方式导致网络在不同阶段的场景共享不足

- UniGen使用统一的模型生成初始位置，初始状态和未来轨迹

问题描述为根据场景信息 R 和初始集合的智能体 S_c ，生成完整的驾驶场景 (R, S) ，也就是所有智能体状态的条件概率
 $p(S|R, S_c)$

符号：

R 表示场景信息；

S_0 为智能体初始状态；

$S_{1..T}$ 为智能体在未来时刻 T 上的轨迹；

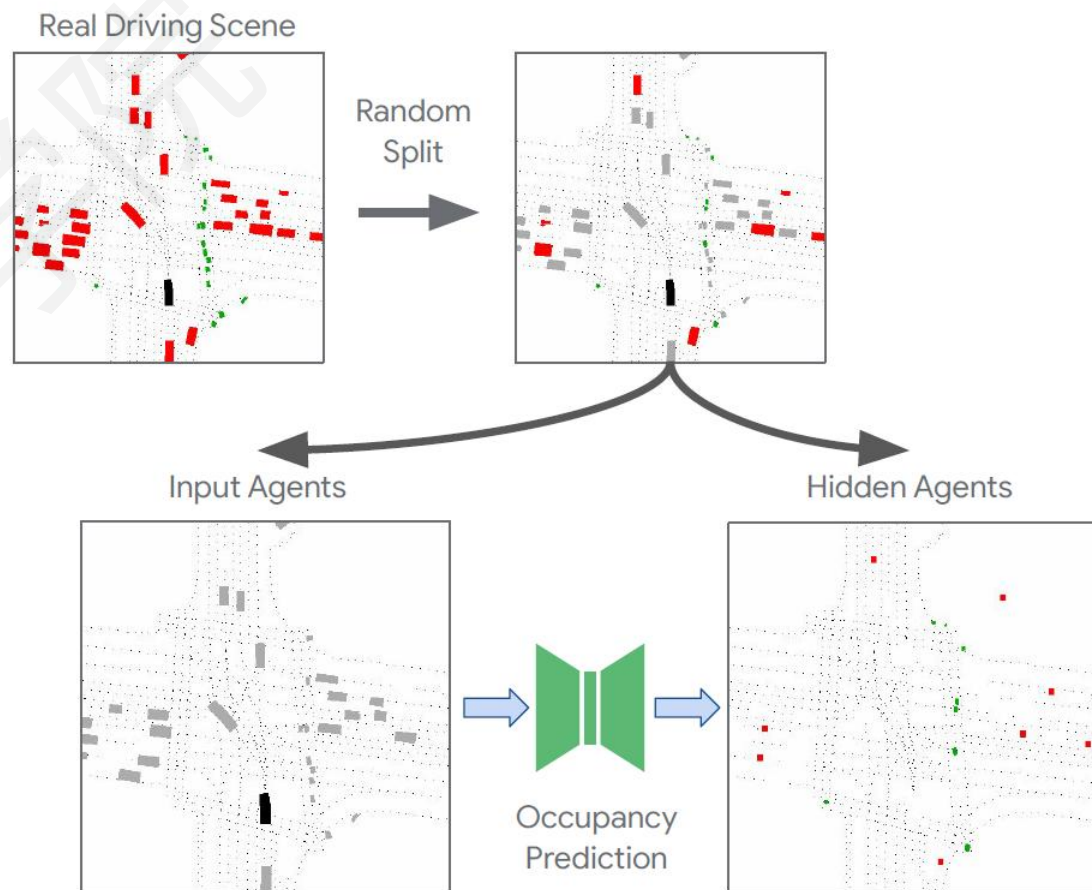
p_ϕ 和 p_ψ 通常为单独训练的概率模型





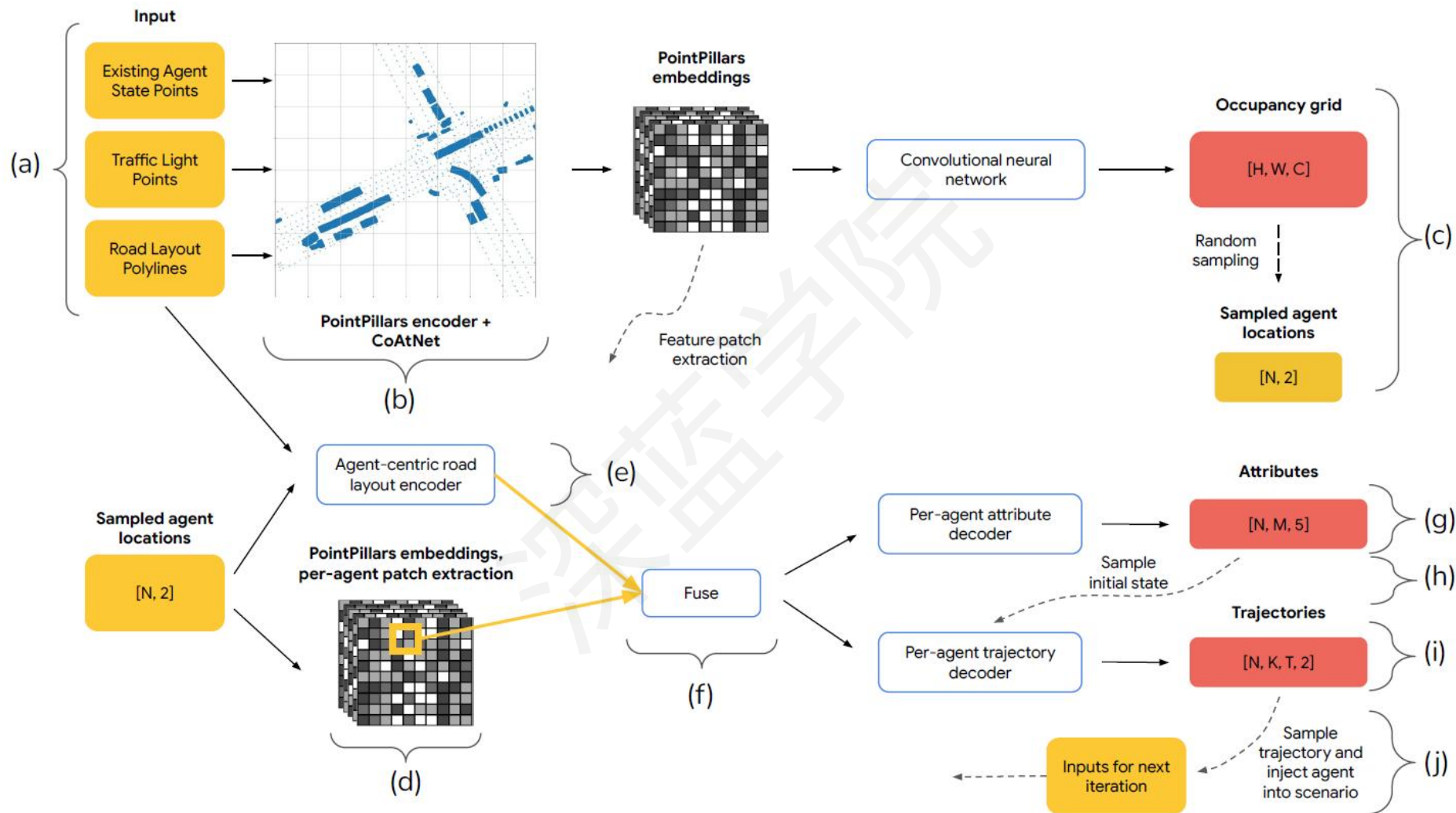
□ 训练数据生成

- 将数据集中的真实智能体分成**输入智能体**和**隐藏智能体**;
- 模型仅根据场景信息和输入智能体, 预测隐藏智能体的位置、属性和轨迹;
- 随机采样概率以控制输入智能体的比例;
- 利用随机遮蔽的方法鼓励模型学习场景运动学。



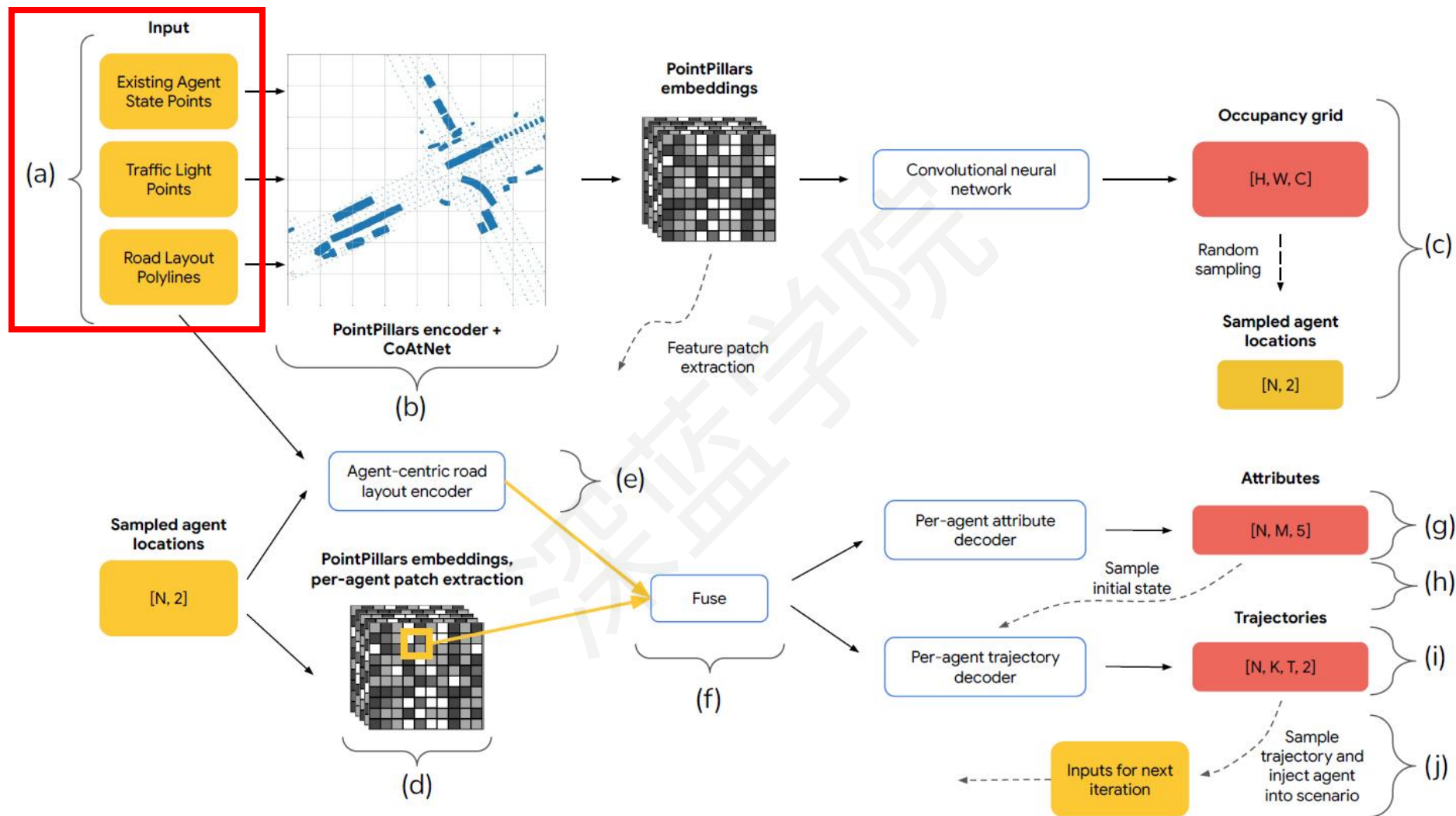


UniGen 驾驶场景生成





UniGen 驾驶场景生成

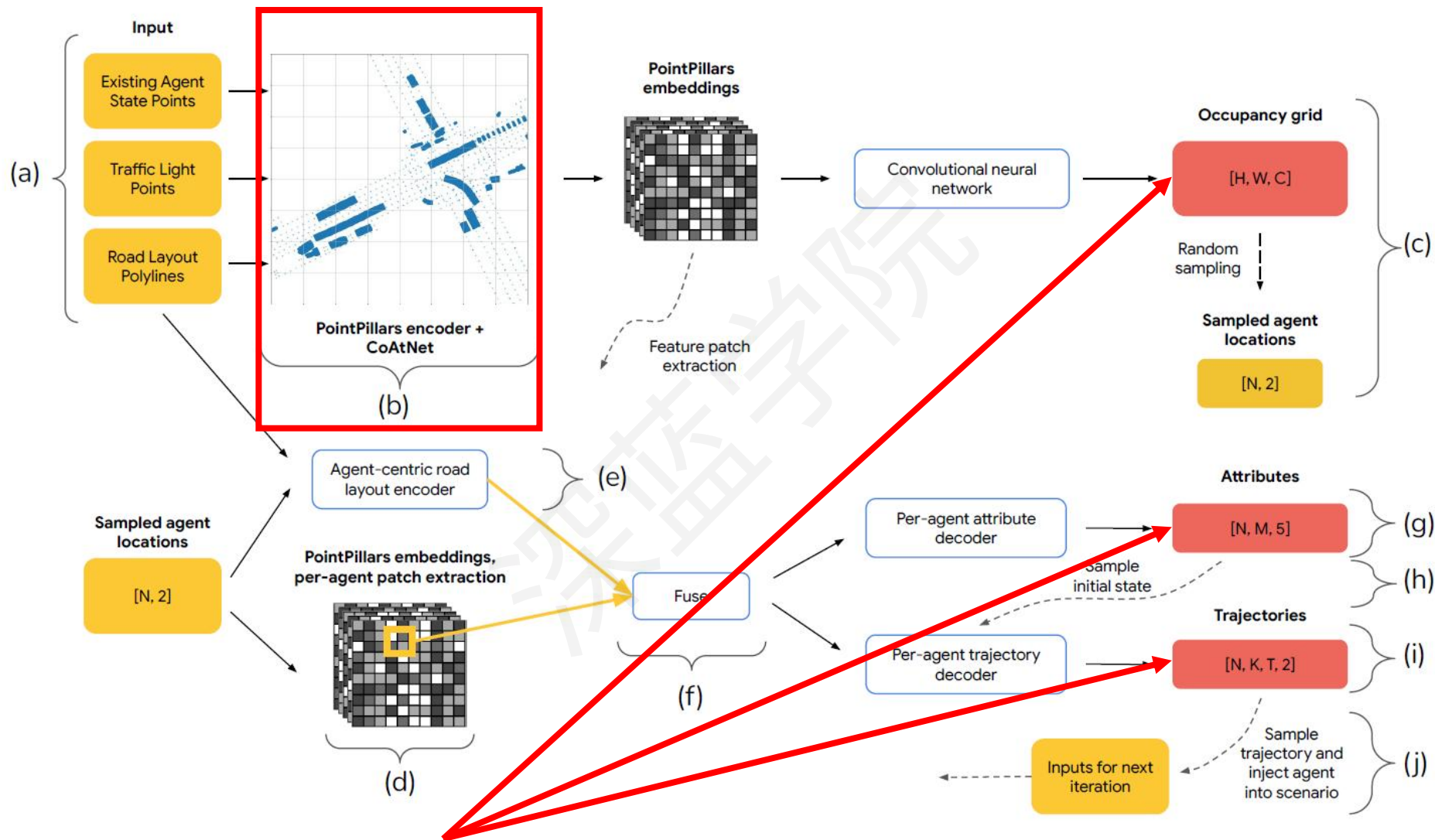


(a)模型输入：道路结构（折线）、交通信号灯（点）、已有智能体bounding box上均匀采样的点。





UniGen 驾驶场景生成

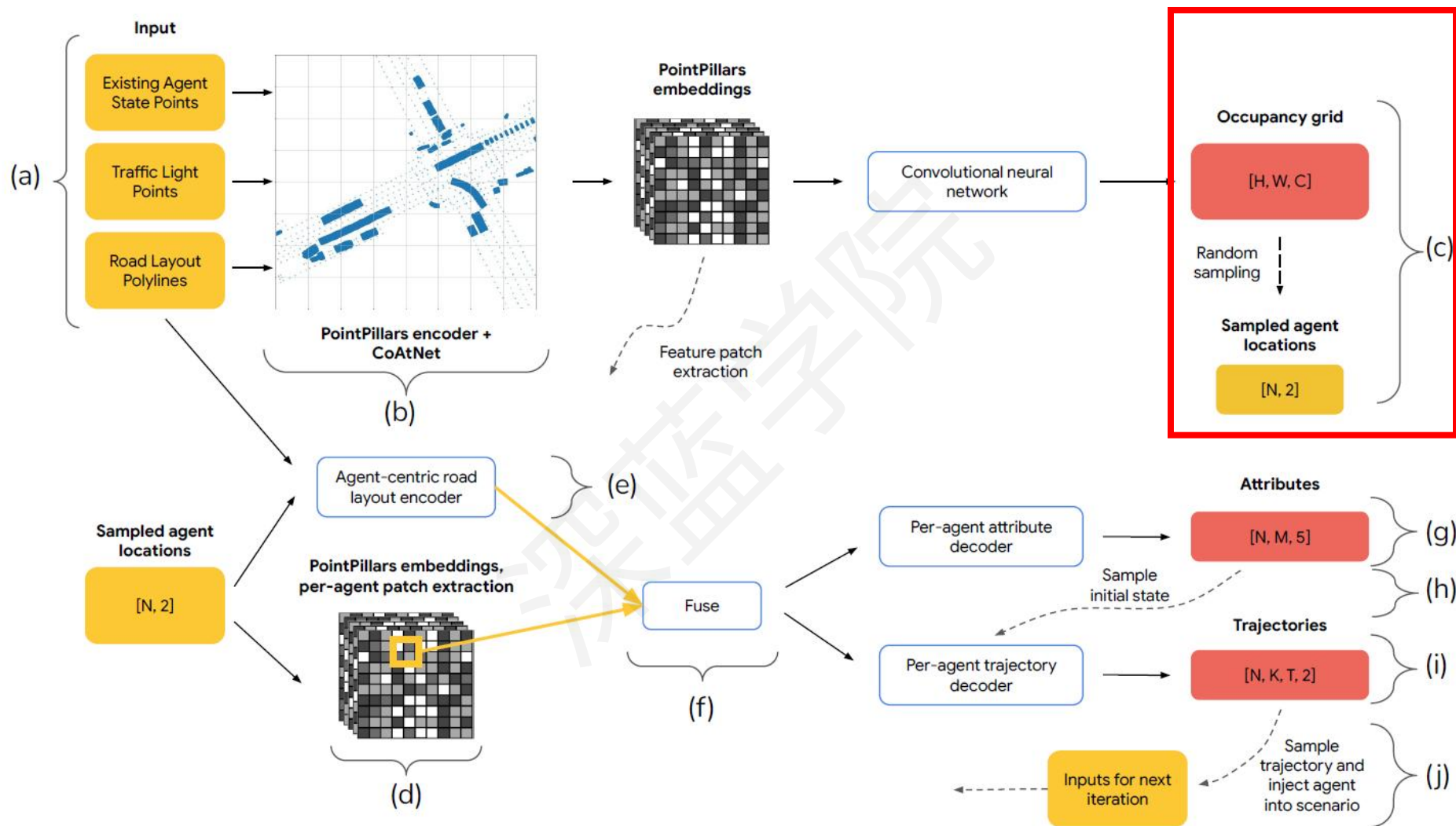


(b)点集被编码为密集的场景嵌入，三个解码器分别预测新智能体插入的占据分布、初始状态和未来轨迹。





UniGen 驾驶场景生成

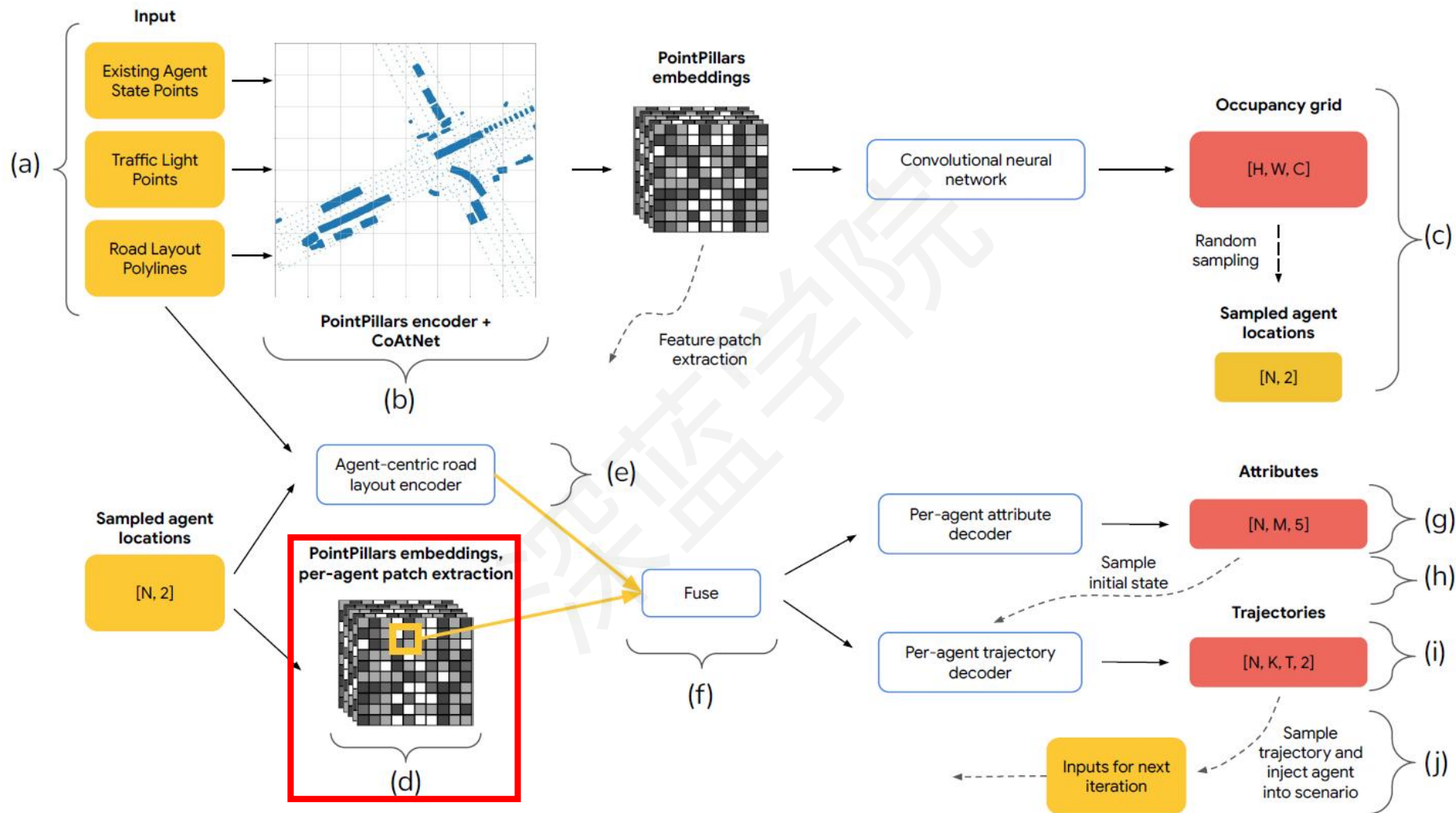


(c) 占据解码器预测 C 个类别智能体的初始位置分布。每轮迭代时从占据热力图中采样一个位置插入智能体。





UniGen 驾驶场景生成

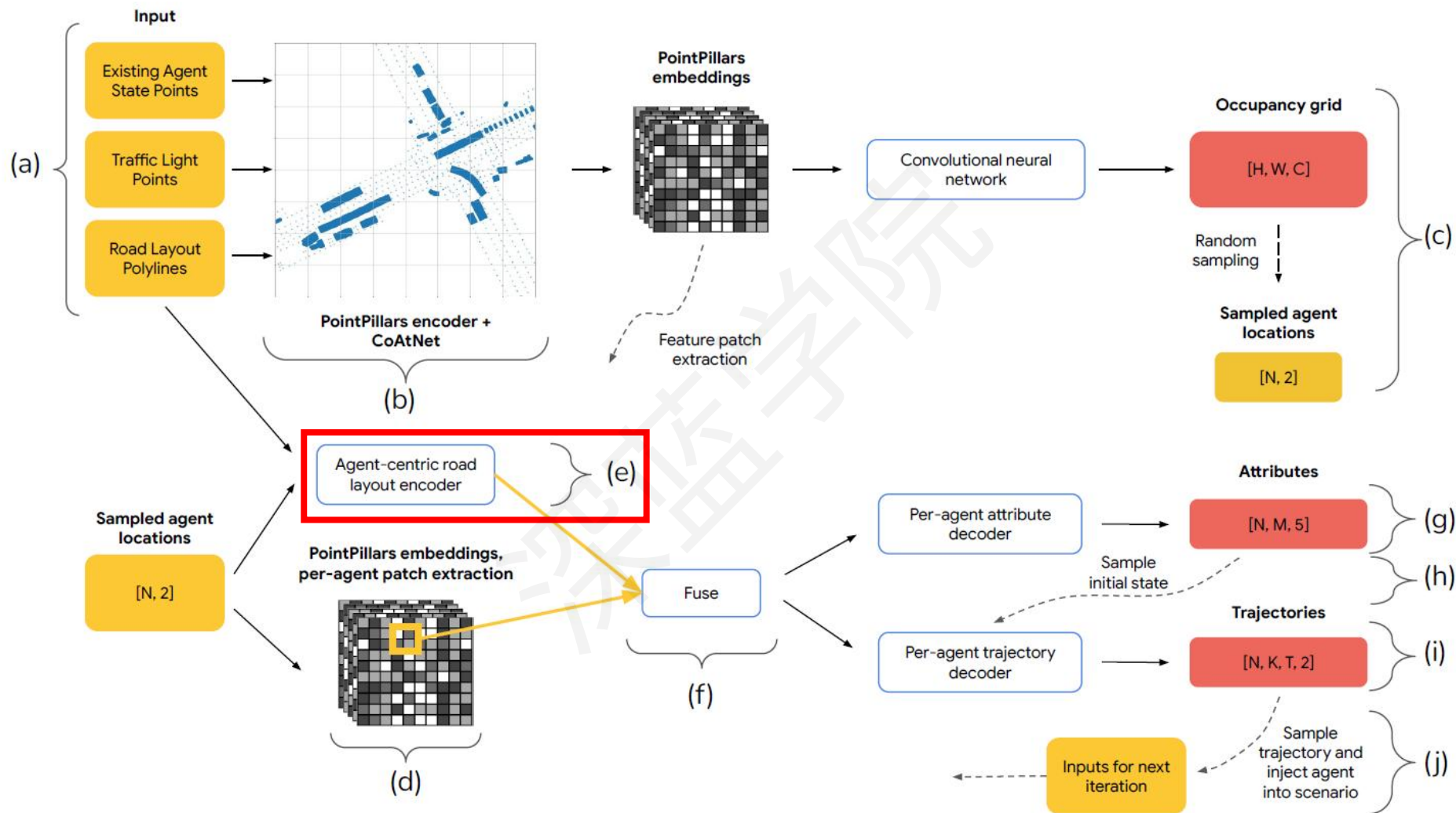


(d) 新智能体的位置通过线性映射到密集场景嵌入的一个位置，提取其周围的特征块。





UniGen 驾驶场景生成

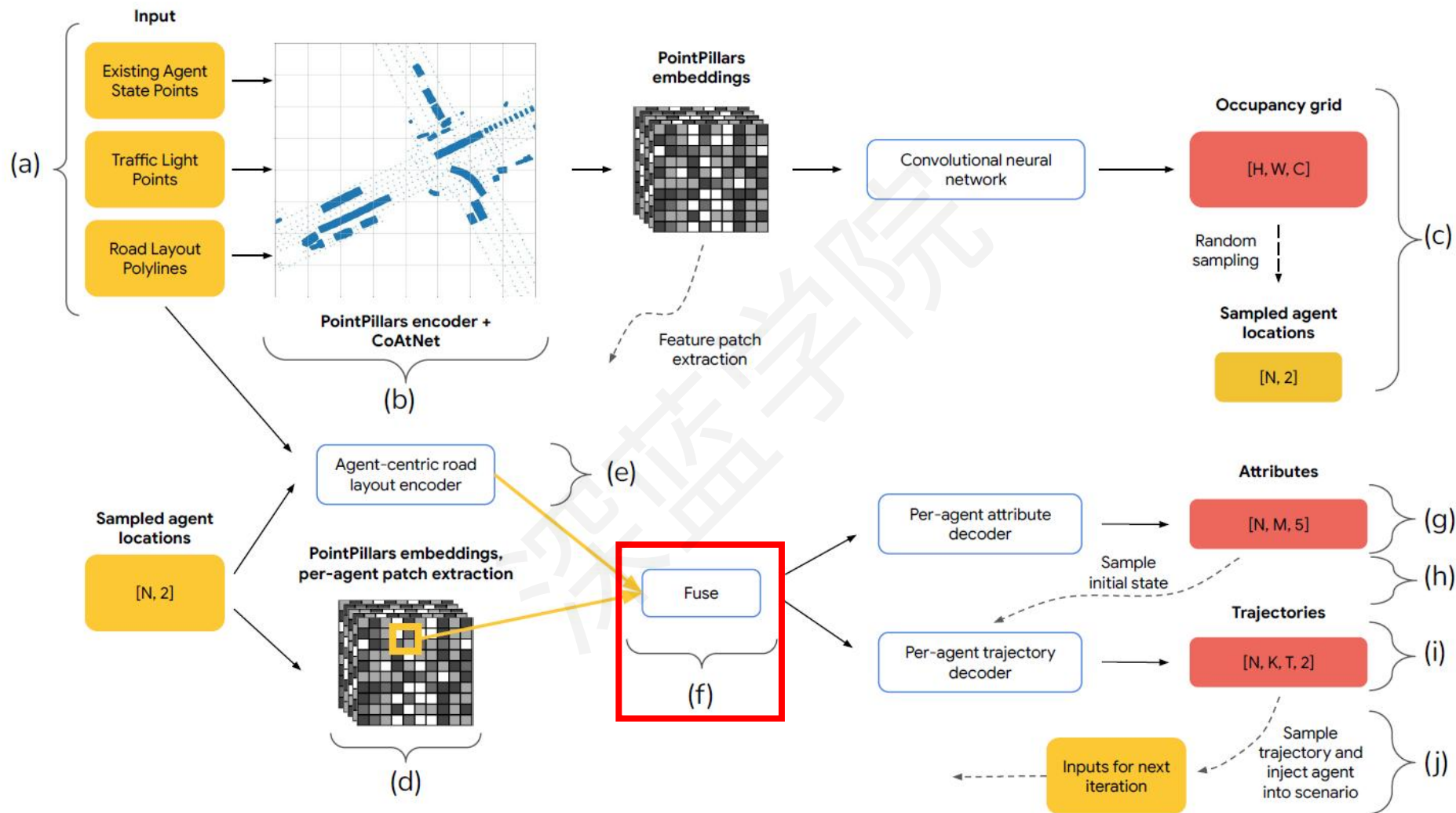


(e) 道路结构transformer提取和编码以智能体插入位置为中心的道路折线。





UniGen 驾驶场景生成

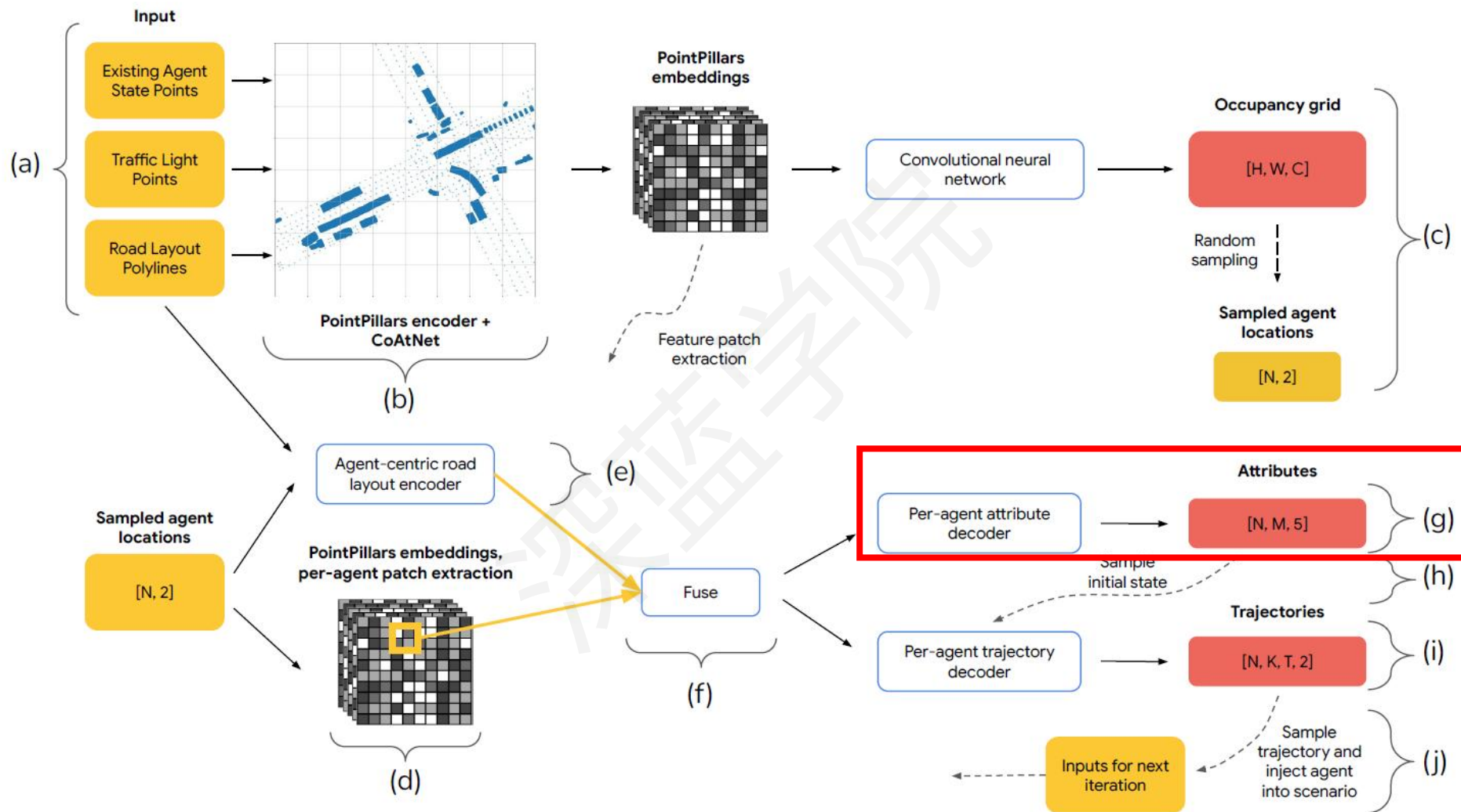


(f) 以新智能体为中心的道路编码通过单层MLP与扁平化的共享场景嵌入中提取的特征块融合。





UniGen 驾驶场景生成

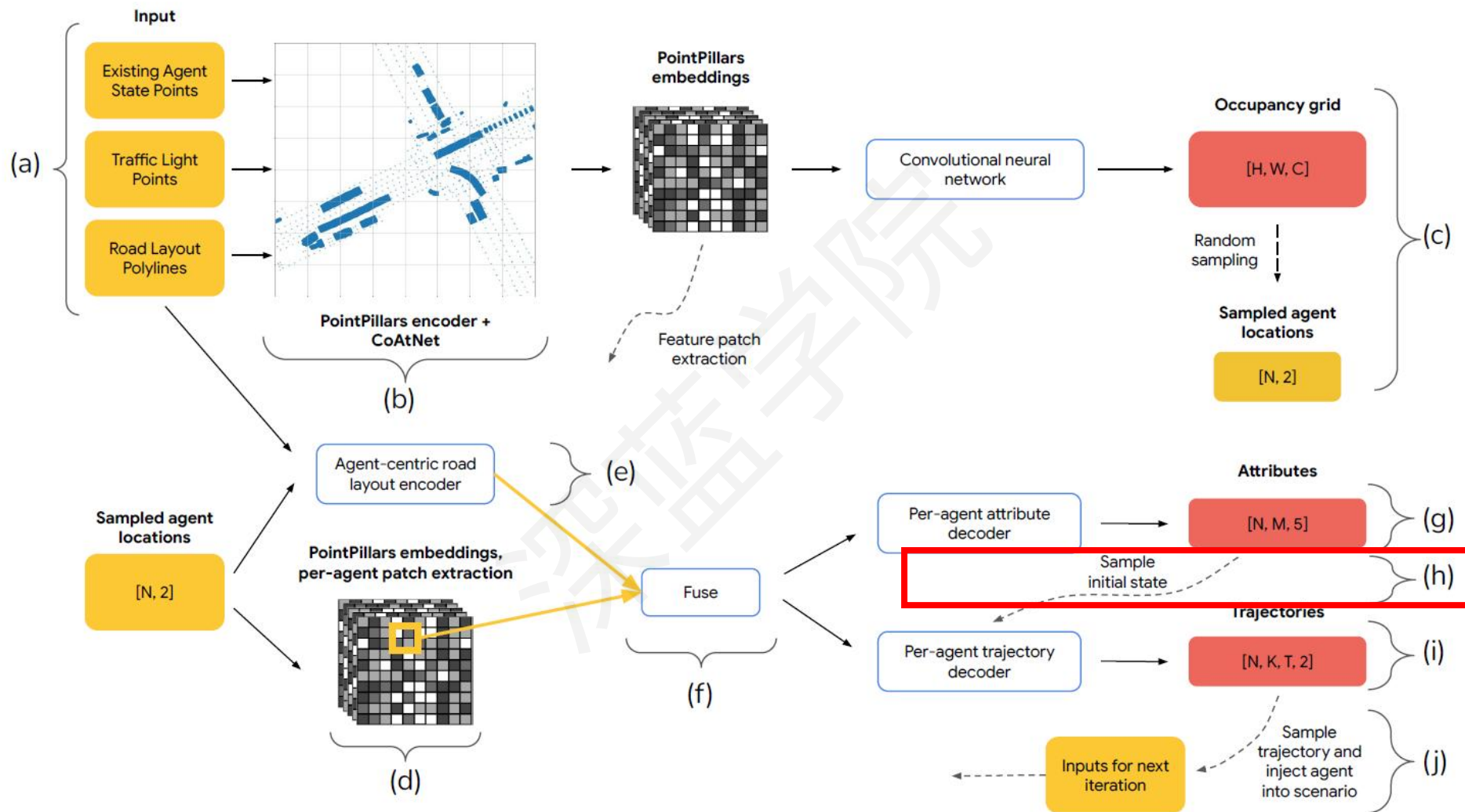


(g) 融合后的特征经过特征解码器预测为具备M个模态的5D多元混合分布（宽度w，长度l，单位方向向量 $\cos(\theta)$, $\sin(\theta)$ ，标量速度）





UniGen 驾驶场景生成

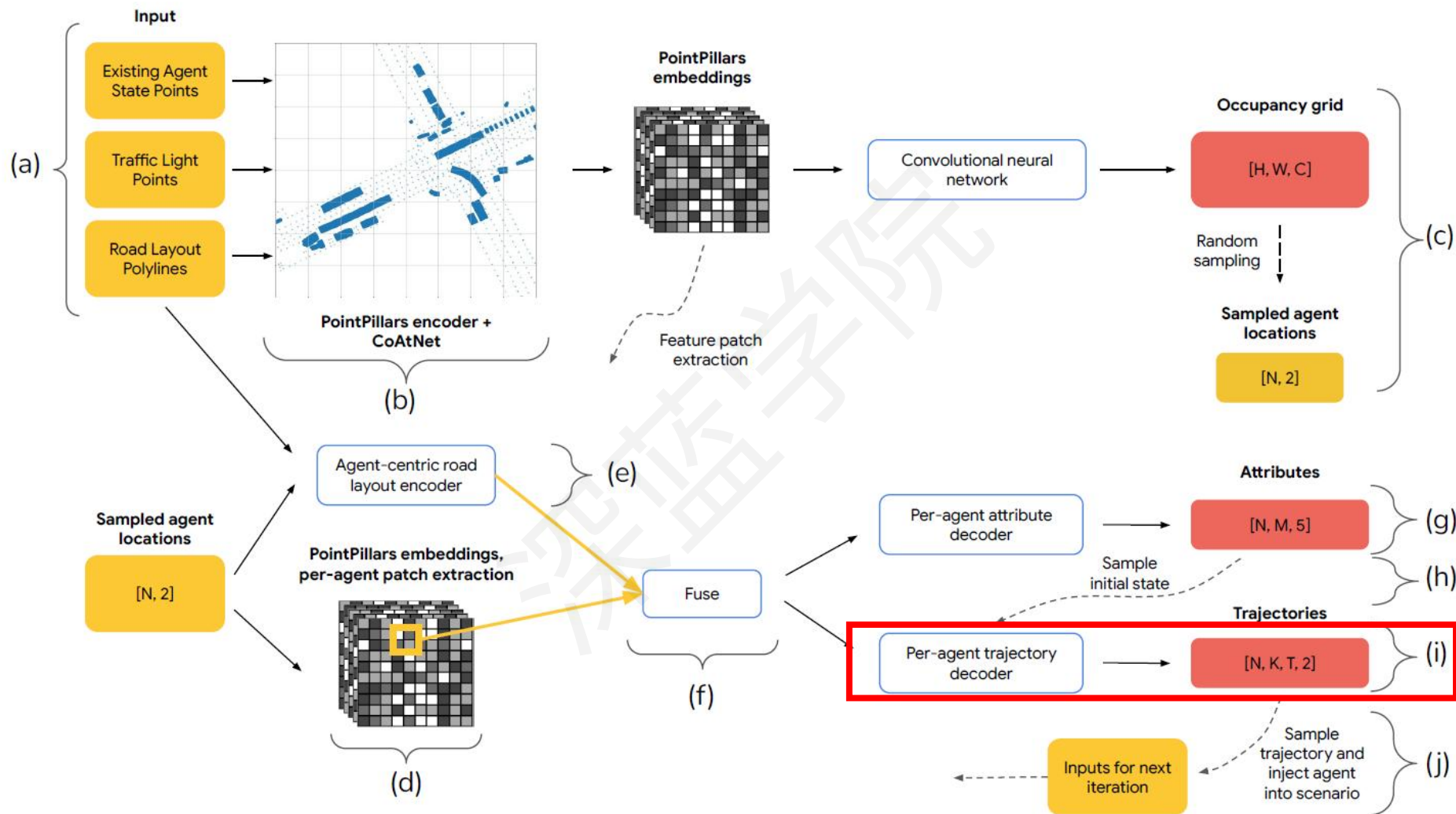


(h) 采样5维的属性数值，和智能体位置构成了完整的初始状态





UniGen 驾驶场景生成

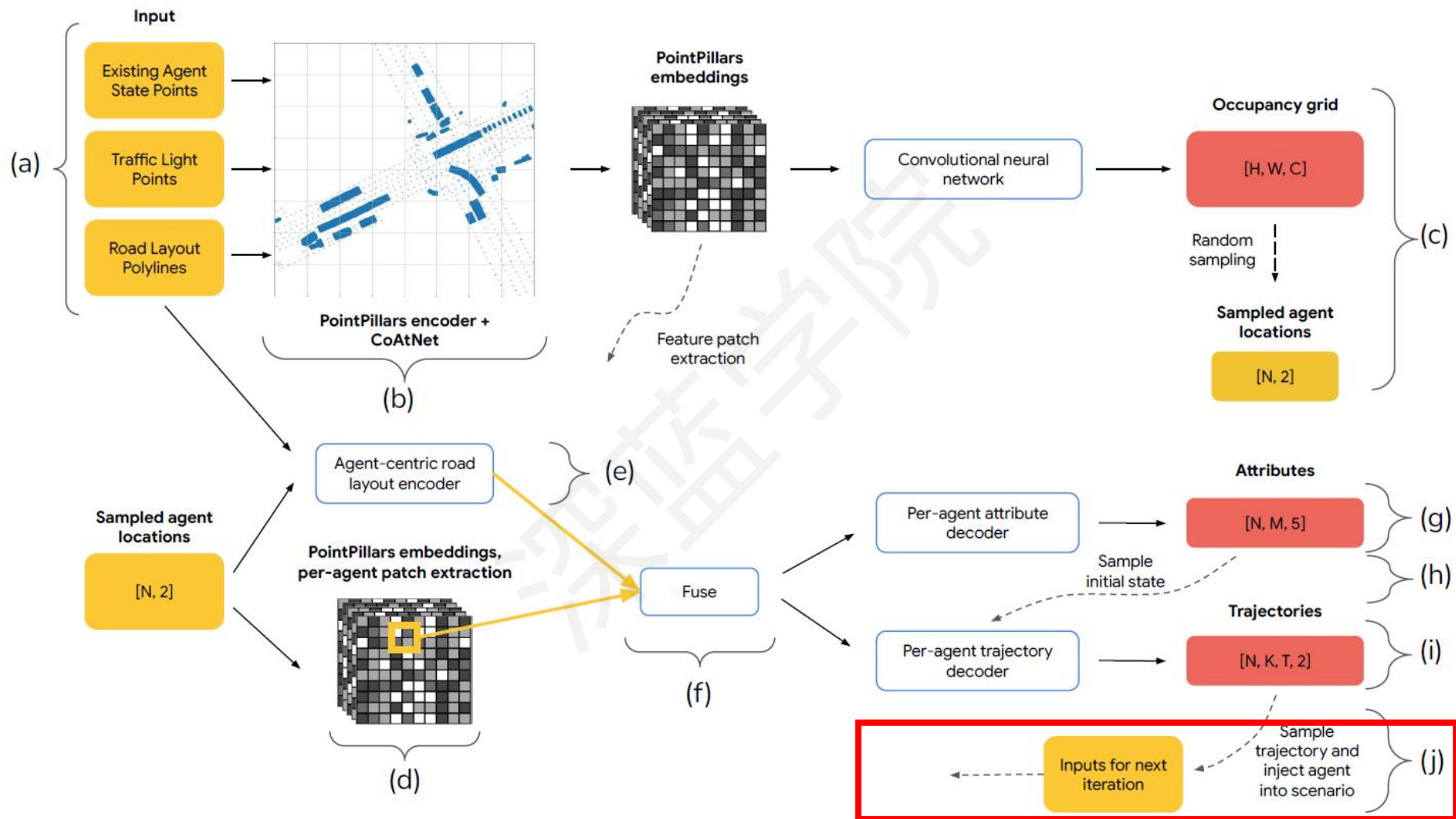


(i) 轨迹解码器收到初始智能体状态和(f)中融合后的特征，预测 K 条时长为 T 的轨迹，每个轨迹点表示为2D高斯分布





UniGen 驾驶场景生成



(j) 从 K 个选择中采样单条轨迹，至此完成了新智能体的构建，新的智能体被加入场景输入(a)，开始新一轮迭代。



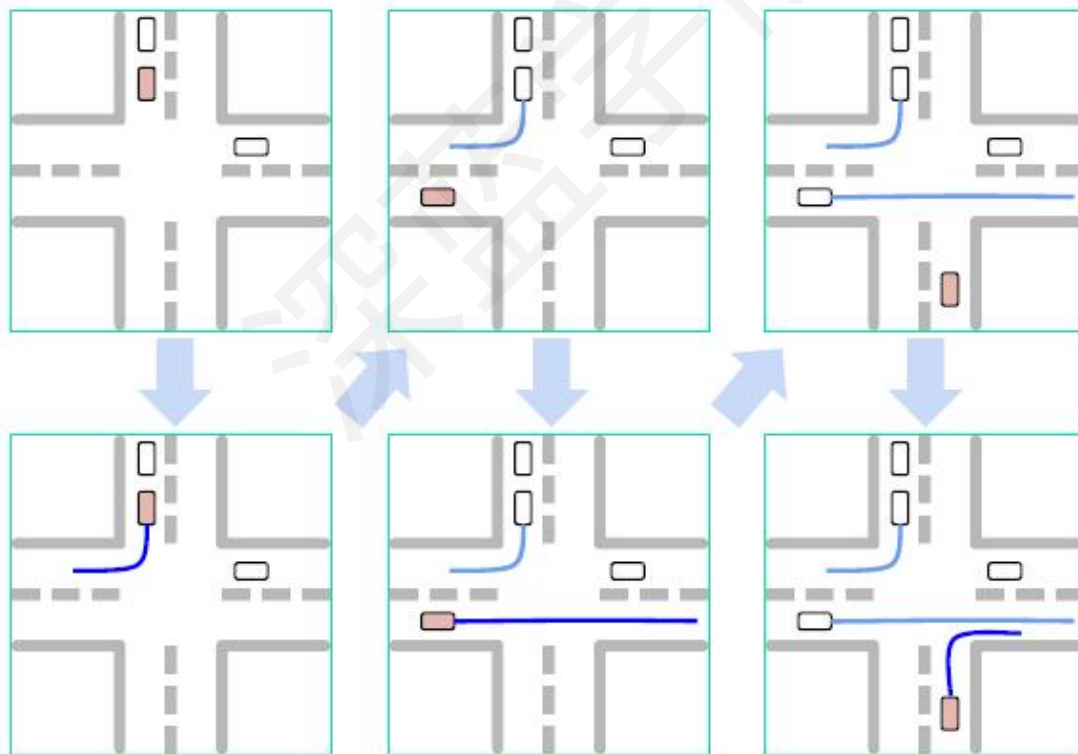


□ 自回归式场景生成

- 在推理时每次插入一个新的智能体，即考虑生成第 i 个智能体的条件分布为

$$P(s^i | R, s^1, s^2, \dots, s^{i-1})$$

- 每次在下一轮迭代中，新生成的智能体将作为模型输入的一部分，影响模型生成的后续智能体的所有属性。
- 相比已有方法，UniGen生成更为接近真实的场景和更低的碰撞率。





UniGen 驾驶场景生成



- 仅根据道路结构模型推理3次生成的不同场景

Ground Truth



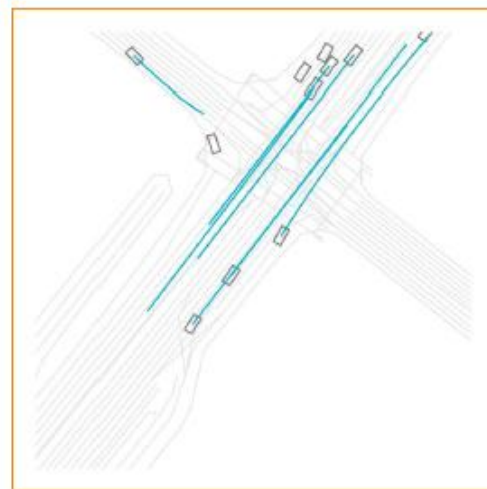
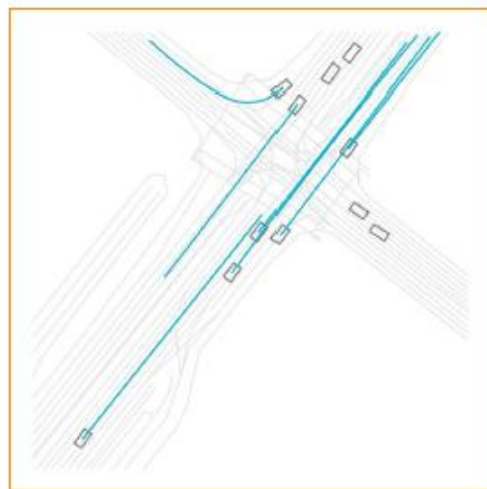
Sample 1



Sample 2



Sample 3





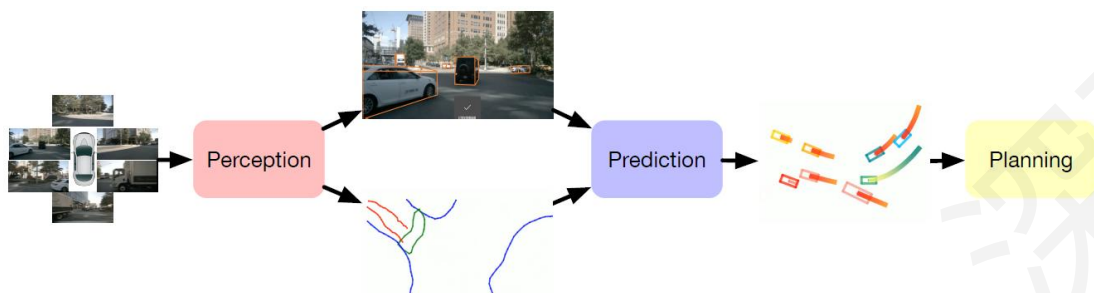
GenAD生成式端到端自动驾驶



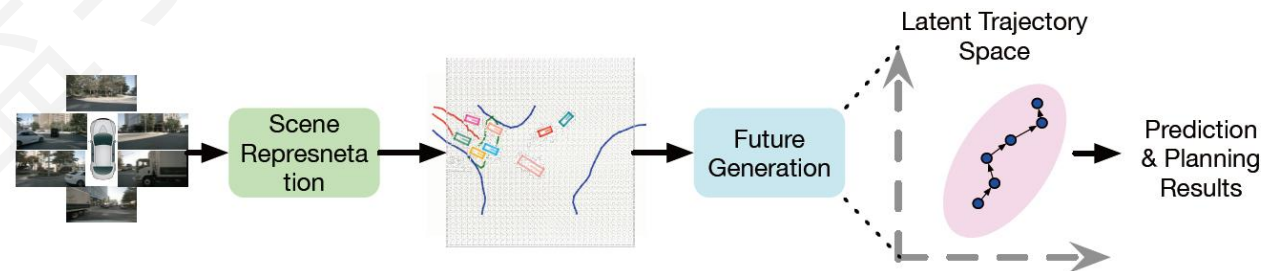
端到端自动驾驶的方法：

- 串行的预测规划设计忽略了未来的车辆交互，无法有效建模高阶的交互；
- 未来的轨迹高度结构化且具有共同的先验（如大多数轨迹都是连续的直线）。

GenAD将端到端自动驾驶建模为轨迹生成问题，使用统一的未来轨迹生成模型同时执行运动预测和规划。



UniAD等方法的设计



GenAD的生成式方法



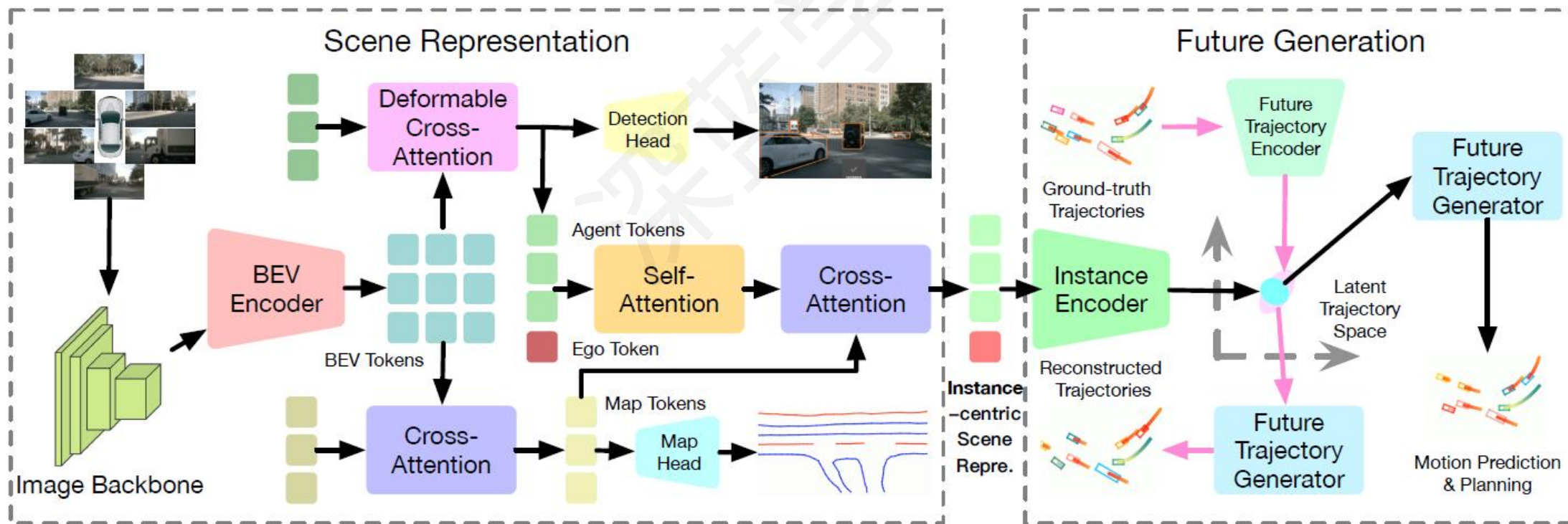


GenAD生成式端到端自动驾驶



□ 系统框架

- 给定周围图像，先后使用图像backbone和BEV编码器得到BEV token;
- 使用交叉注意力和可变形交叉注意力得到map token和agent token;
- 通过额外的ego token，经过自注意力和交叉注意力得到实例中心的场景表征;
- 将场景表征映射到结构化的轨迹潜空间;
- 最后使用轨迹生成器生成包含预测和规划的未来轨迹。





GenAD生成式端到端自动驾驶



实例中心的场景表征

图像 → BEV

使用卷积神经网络和特征金字塔得到多尺度图像特征 F ，初始化BEV token作为查询 B_0 ；使用可变形注意力 $DA(Q, K, V)$ 得到BEV特征 B ：

$$B = DA(B_0, F, F)$$

BEV → 地图

使用全局交叉注意力机制 $CA(Q, K, V)$ 更新初始化的地图查询 M_0 ：

$$M = CA(M_0, B, B)$$

BEV → 智能体

类似的，使用可变形注意力获得agent token：

$$A = DA(A_0, B, B)$$

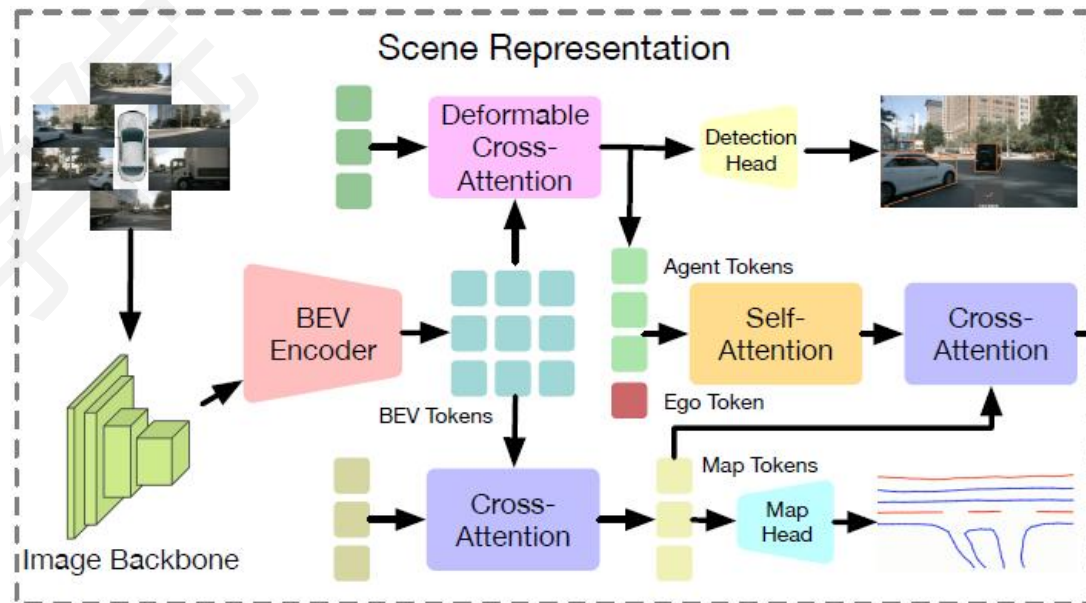
场景表征

在学习到的agent token A 上增加自车的token e 得到instance token I ：

$$I = \text{concat}(e, A)$$

利用自注意力 $SA(Q, K, V)$ 学习自车和其他智能体的交互： $I \leftarrow SA(I, I, I)$

准确的预测还需要地图信息，因此最终通过交叉注意力更新instance token I ： $I \leftarrow CA(I, M, M)$





□ 轨迹建模及轨迹生成

- 将规划问题建模为根据场景编码输出 $\mathbf{I}$ 的带有不确定性的轨迹生成问题，即

$$\mathbf{T} \sim p(\mathbf{T}|\mathbf{I})$$

其中， $\mathbf{T}(T, f) = \{\mathbf{w}^{T+1}, \mathbf{w}^{T+2}, \dots, \mathbf{w}^{T+f}\}$ 为当前时刻 T 至未来 f 帧的轨迹。

- 考虑到自车和其他智能体的轨迹具备高度的结构化特征（例如大多数轨迹是匀速直线），使用VAE学习隐空间 $\mathbf{Z}$ 的轨迹先验。使用真值轨迹编码器 e_f 将未来轨迹映射为隐空间 $\mathbf{Z}$ 的高斯分布，

$$p(\mathbf{z}|\mathbf{T}(T, f)) \sim N(\mu_f, \sigma_f)$$

为了考虑不同时刻车辆的时间关系，将联合分布分解为，

$$p(\mathbf{T}(T, f)|\mathbf{z}^T) = p(\mathbf{w}^{T+1}|\mathbf{z}^T) \cdot p(\mathbf{w}^{T+2}|\mathbf{w}^{T+1}, \mathbf{z}^T) \dots p(\mathbf{w}^{T+f}|\mathbf{w}^{T+1}, \dots, \mathbf{w}^{T+f-1}, \mathbf{z}^T)$$

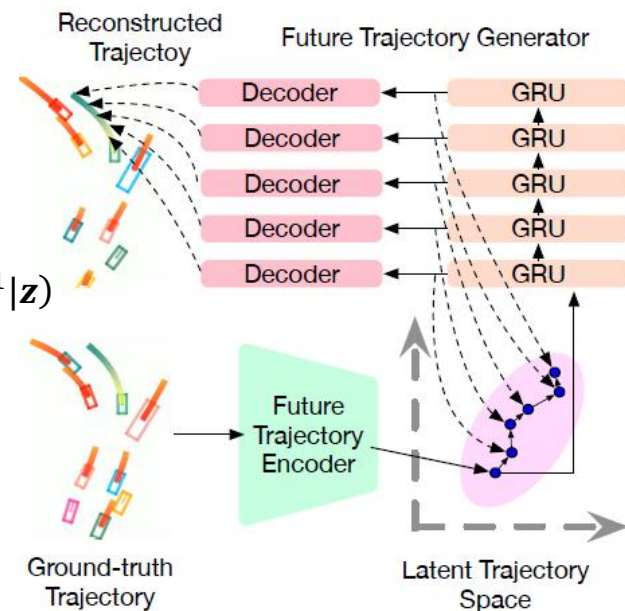
- 轨迹生成时，在高斯分布 $N(\mu_f, \sigma_f)$ 采样当前时刻的隐状态 $\mathbf{z}^T$ ，使用基于MLP的解码器 d_w 实例化 $p(\mathbf{w}^{T+1}|\mathbf{z})$

$$\mathbf{w} = d_w(\mathbf{z})$$

使用GRU模型 g 生成未来轨迹，每次输入当前隐式表征 $\mathbf{z}_t$ ，并将其输出下个状态，

$$g(\mathbf{z}_t) = \mathbf{z}_{t+1}$$

随后通过解码器获得下个路点 $\mathbf{w}^{t+1} = d_w(\mathbf{z}_{t+1})$ 。





GenAD生成式端到端自动驾驶



训练时，额外加入分类解码器预测智能体的类别 $\hat{c}_a$ ，训练轨迹编码器、生成器、路点解码器，定义损失项为：

$$J_{prior} = L_{tra}(\hat{T}_e, T_e) + \frac{1}{N_a} L_{tra}(\hat{T}_a, T_a) + \lambda_c L_{focal}(\hat{C}_a, C_a)$$

其中， L_{tra} 为包含L1范数和碰撞约束的轨迹损失， $\hat{T}_e, T_e, \hat{T}_a, T_a$ 分别为自车及他车的真值和预测轨迹， C_a 为智能体的类别。

使用KL散度损失监督实例的分布 $p(\mathbf{z}|\mathbf{I})$ 和真值分布 $p(\mathbf{z}|\mathbf{T}(T, f))$

$$J_{plan} = D_{KL}(p(\mathbf{z}|\mathbf{I}), p(\mathbf{z}|\mathbf{T}(T, f)))$$

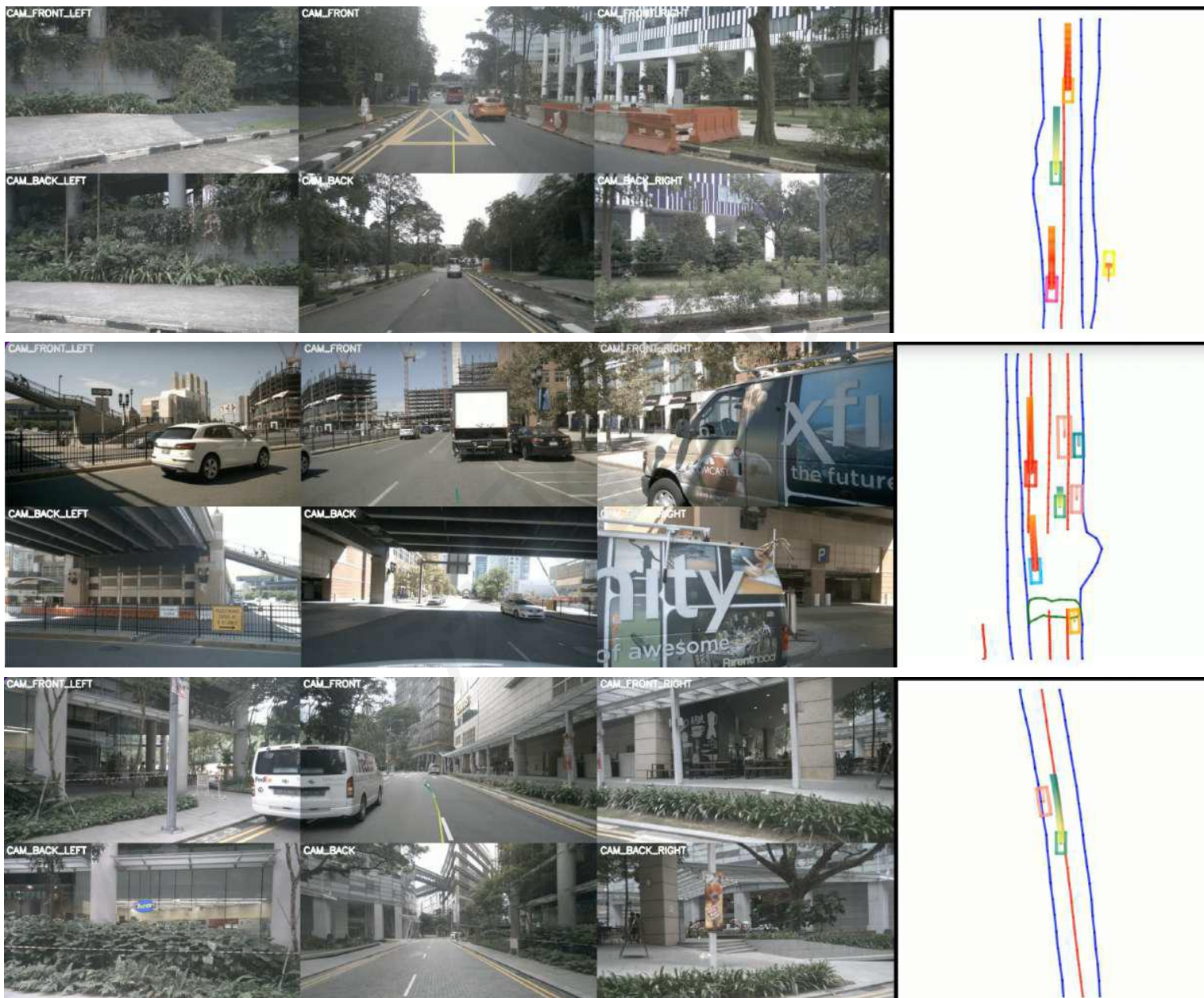
总损失还包括地图损失 J_{map} 和物体检测损失 J_{det}

$$J_{GenAD} = J_{prior} + \lambda_{plan} J_{plan} + \lambda_{map} J_{map} + \lambda_{det} J_{det}$$





GenAD生成式端到端自动驾驶





总结：生成模型



- 生成模型：扩散模型，GAN，自回归网络，VAE...
- 发展趋势
 - 生成更多驾驶风格的轨迹
 - 解决长尾场景
 - 闭环测试
- 难点
 - 轨迹的多模态学习
 - 学习智能体之间的交互
 - 如何高效生成类人轨迹

深蓝学院





大纲



生成模型

模仿学习

强化学习

深蓝学院

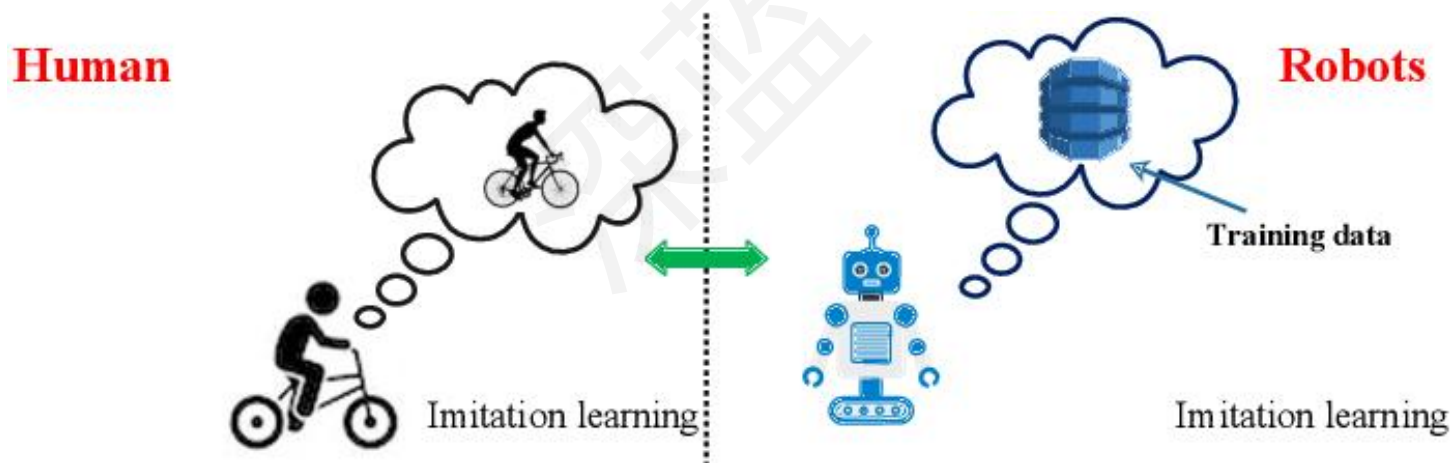




模仿学习

□ 什么是模仿学习?

- 通过观察和模仿专家的行为来学习任务
- 核心思想：通过专家示例（驾驶员的示教数据）来训练自动驾驶系统，使其能够在相似的环境中做出类似的决策



□ 背景：

- 传统自动驾驶系统依赖物体检测的方法在场景中观测其他智能体，这种物体检测假设物体是离散集合，而且缺少不确定性，会导致预测智能体行为时误差增大；
- 当使用占用栅格地图计算可行空间时，为整个场景预测一个栅格浪费资源，因为只有特定时空域在自动驾驶车辆范围内或与其相关。

□ 解决方案：

- 在相关时空点使用决策者查询栅格，限制在相关区域的计算量；
- 利用这种表征，针对避免碰撞、舒适度、安全性和可解释性等关键因素，对候选轨迹进行评估。





问题定义:

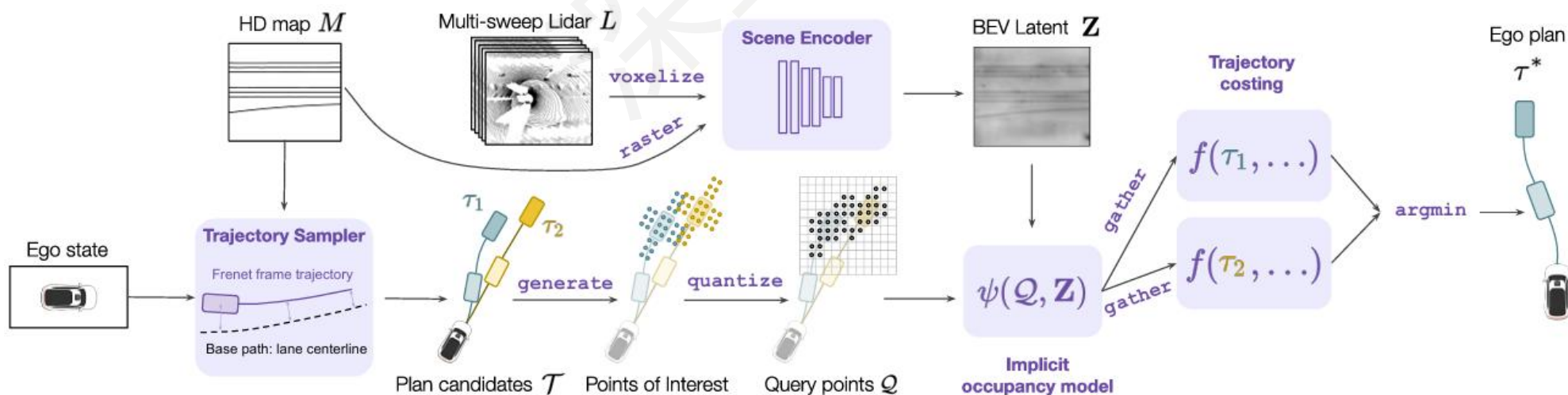
- 目标: 给定雷达观测 L 和高精地图 M 的短时历史信息, 找到可执行的最好轨迹规划。
- 每当有新的传感器观测, QuAD通过优化目标 $\tau^* = \arg \min_{\tau \in \mathcal{T}} J(\tau, \mathbf{Z}, M)$ 规划轨迹, 其中 $\mathcal{T}$ 是候选轨迹的有限集, $\mathbf{Z} \in \mathbb{R}^{H \times W \times C}$ 是由可学习编码器从中提取的环境BEV的潜在表征。

- J 定义为 N 个可解释代价函数 f_i 线性组合, 权重为可学习的系数 w_i :

$$J(\tau, \mathbf{Z}, M) = f(\tau, \psi(Q_\tau, \mathbf{Z}), M)$$

$$= \sum_i^N w_i f_i(\tau, \psi(Q_\tau^{(i)}, \mathbf{Z}), M)$$

- ψ 可以在任何连续的时序位置 Q 下查询占用, $\psi(\mathbf{q}, \mathbf{Z})$ 输出在BEV中连续时序位置 $\mathbf{q} = (x, y, t)$ 在某物体表面的概率。





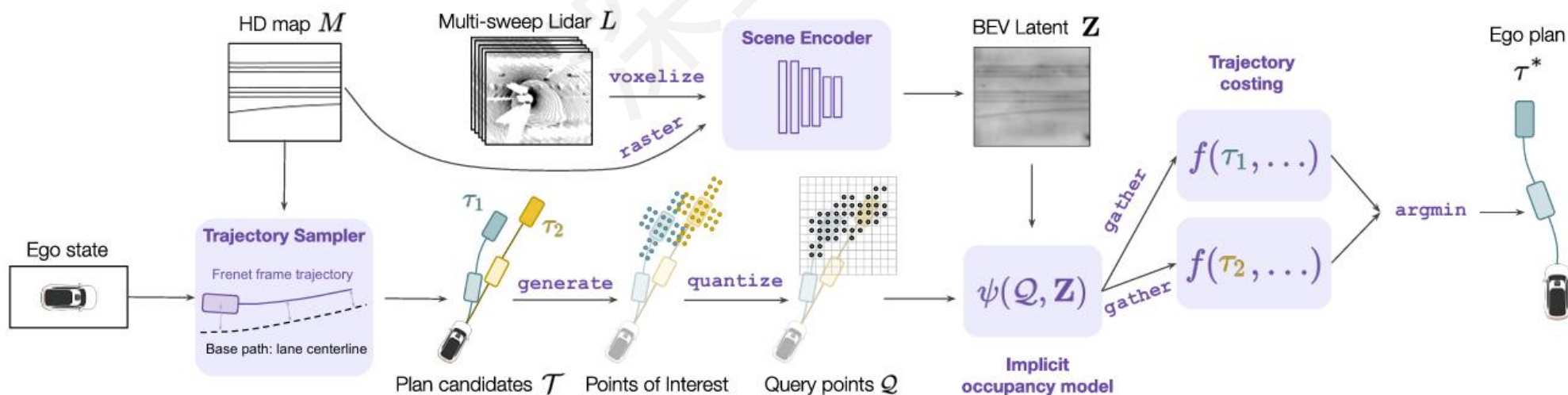
□ 路径采样及查询点定义

路径采样:

- 持续5s;
- 自车状态包括: BEV下的位置、速度、转向角;
- 在Frenet坐标系下采样横纵集合。

关注点:

- 目的: 可以在整个候选规划过程中可以围绕自车覆盖相关区域;
- 以一定的分辨率在自车bounding box内采样一个统一的点网格, 并简单地向前/向后移动ego车辆的长度, 向右/向左移动宽度, 以获得所有感兴趣的点。



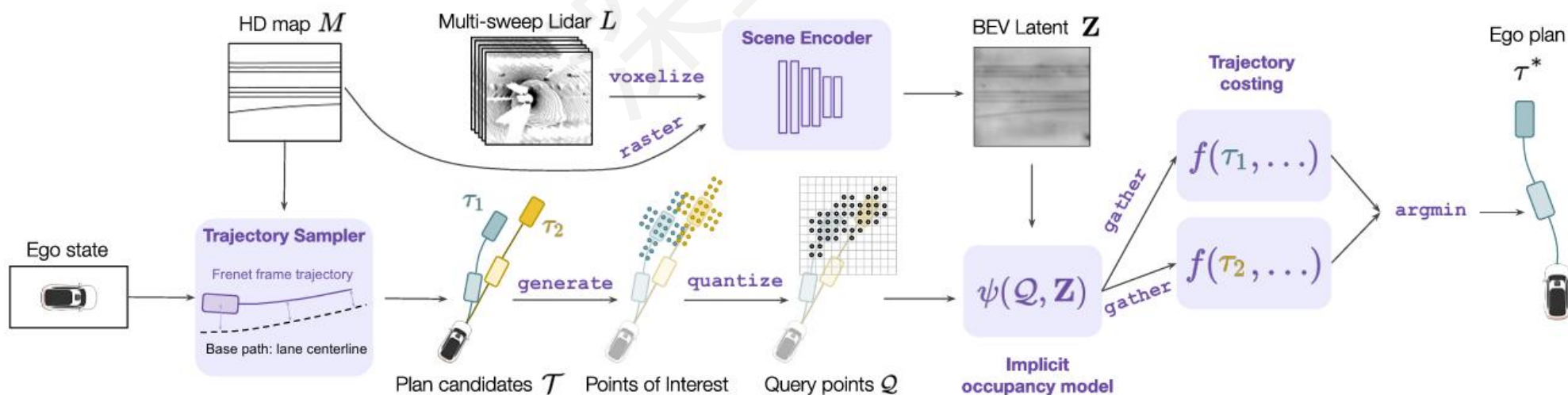
□ 隐式占用模型:

场景编码:

- 输入5个最新扫描的雷达点云信息, 每个扫描包括SDV坐标下 (p_x, p_y, p_h) 的集合点;
- 将雷达信息在BEV下体素化为3D tensor;
- 在高精地图下光栅化道路中线的polyline作为BEV二进制地图, 然后将其传入Feature Pyramid Network得到融合BEV特征地图 $\mathbf{Z}$ 。

隐式占用解码:

- 在查询点 $\mathcal{Q} = \{\mathbf{q}_j\}_{j \in [1, |\mathcal{Q}|]}$ 集合预测占用概率, 每个查询点 $\mathbf{q} = (x, y, t) \in \mathbb{R}^3$ 表示在未来时间 t 下BEV中的时序点。





□ 轨迹代价

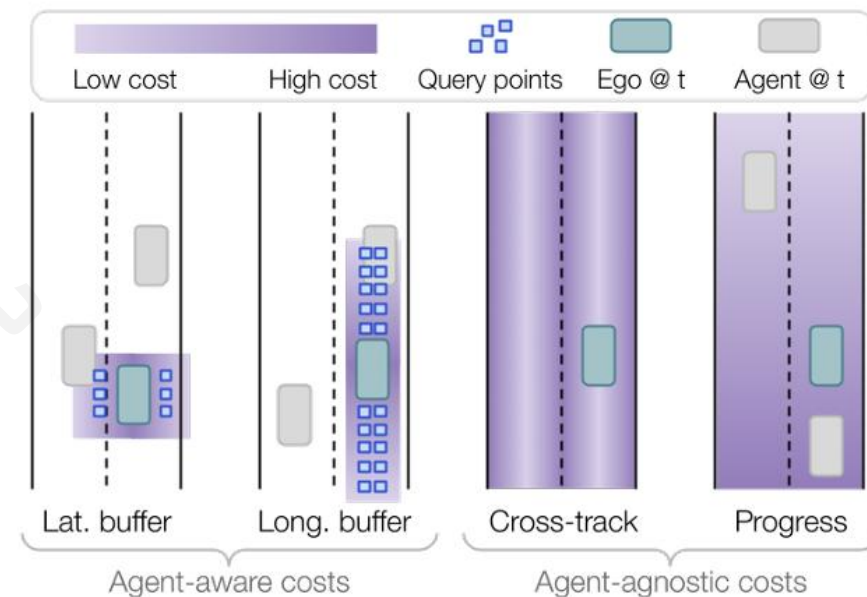
f_i 包括 agent-agnostic 代价和 agent-aware 代价:

- agent-agnostic 代价描述舒适度、规则服从度和候选轨迹到目标点的完成度;
- agent-aware 代价计算和其他智能体相关的轨迹安全性。

□ 网络学习

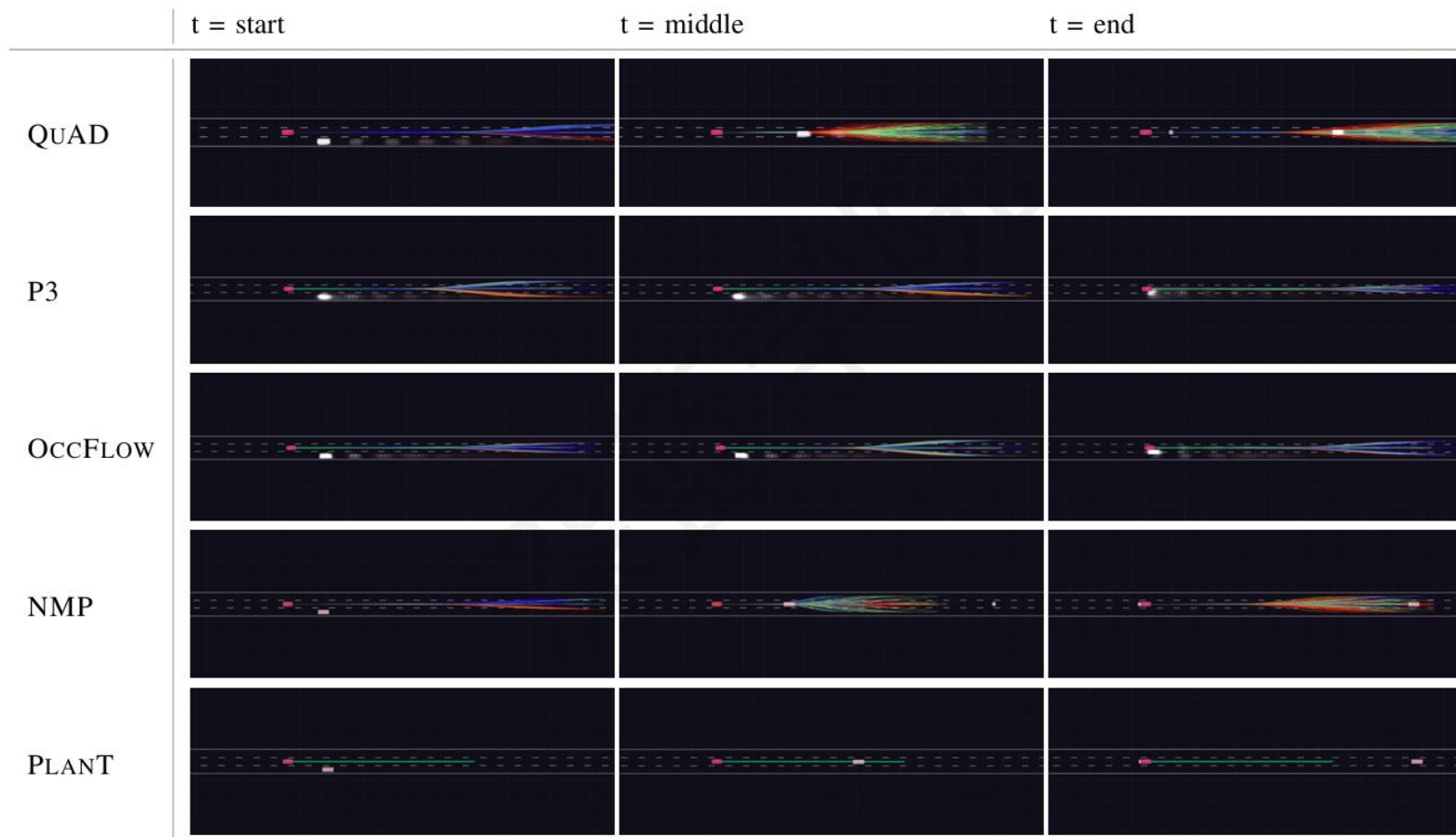
两个阶段:

- 首先, 训练隐式占用模型学习感知和预测;
- 然后, 固定占用模型, 训练代价集合权重 $\{w_i\}$ 来模仿专家驾驶员。



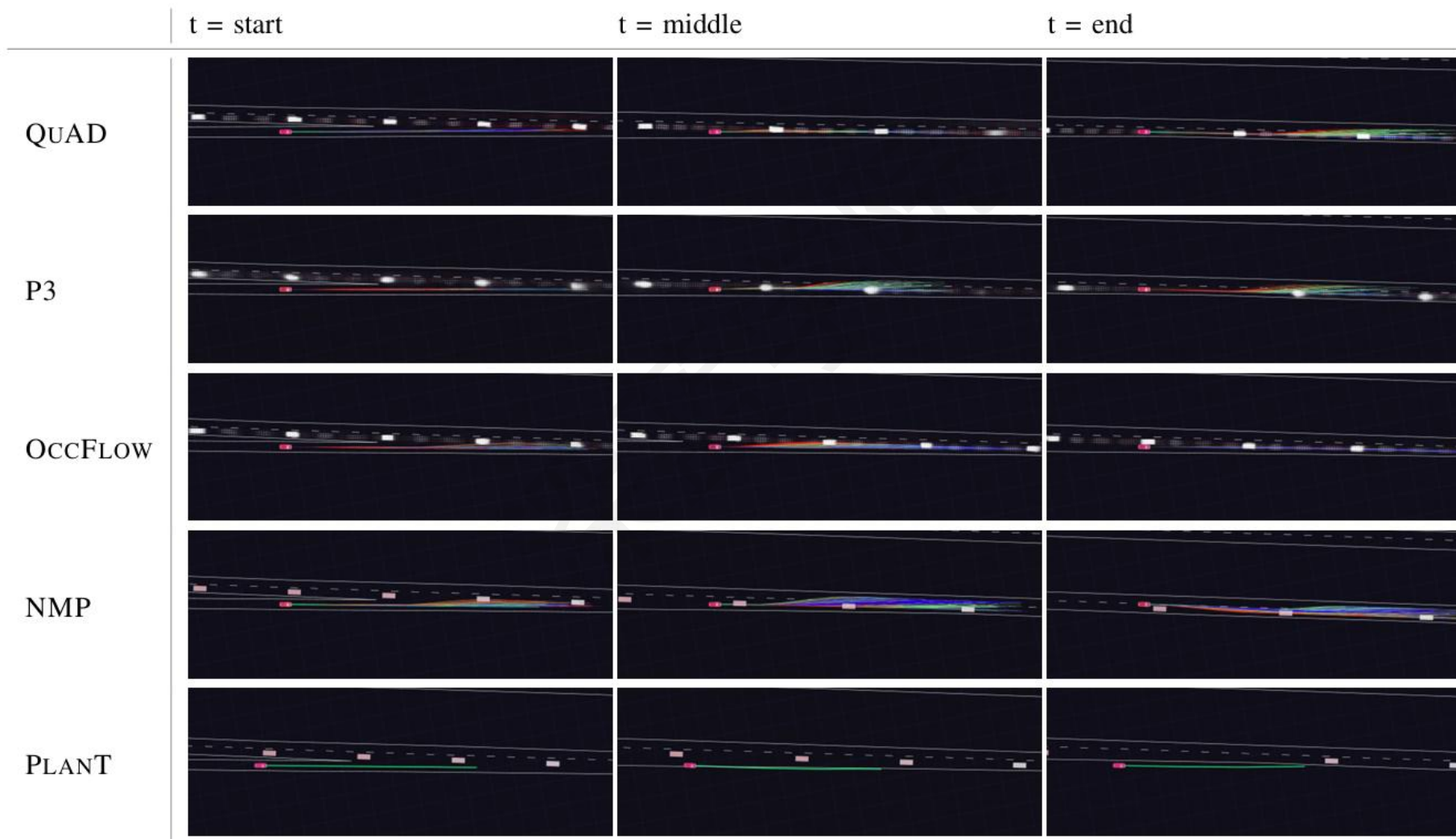


效果对比



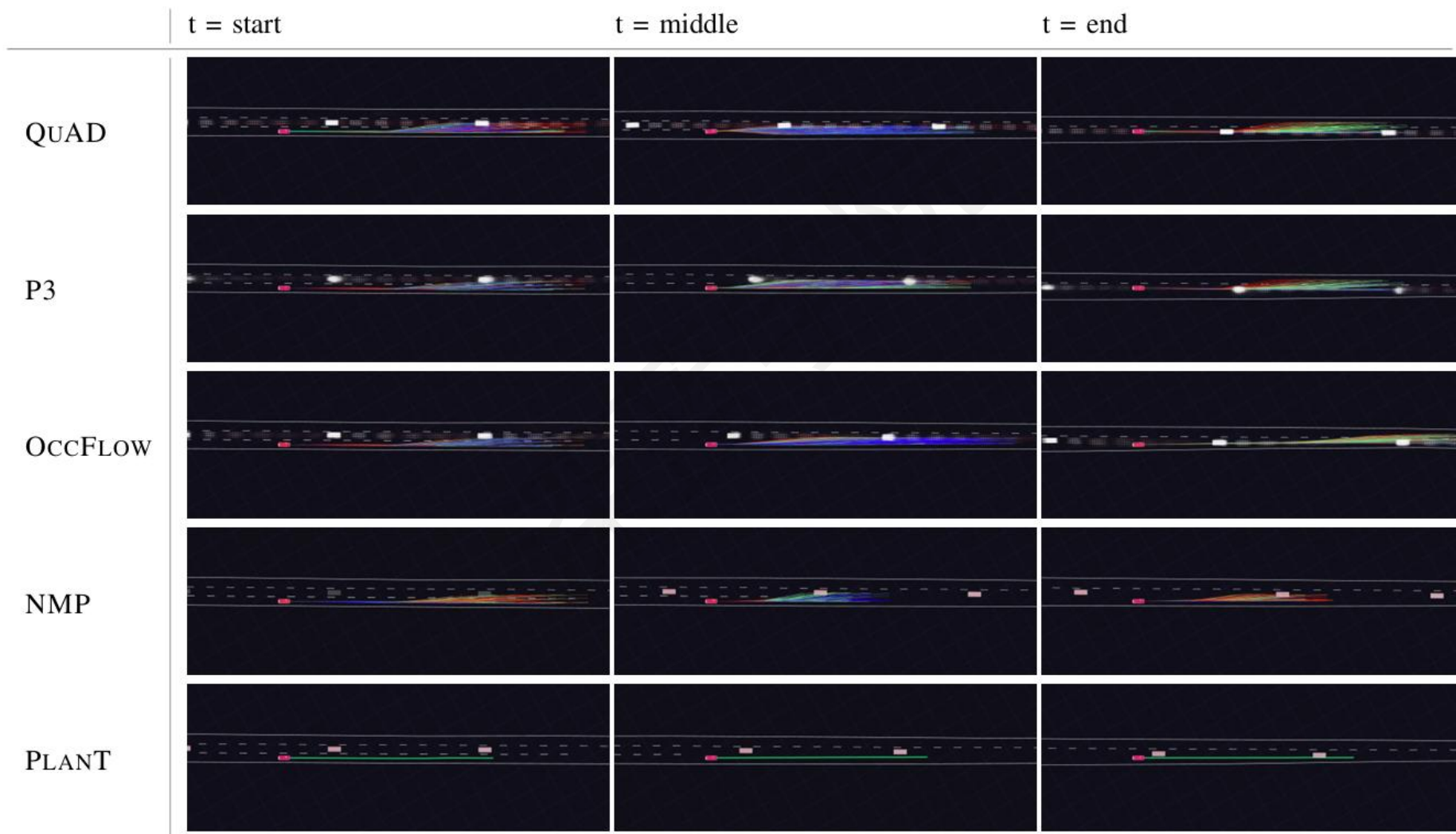


效果对比





效果对比



□ 特点

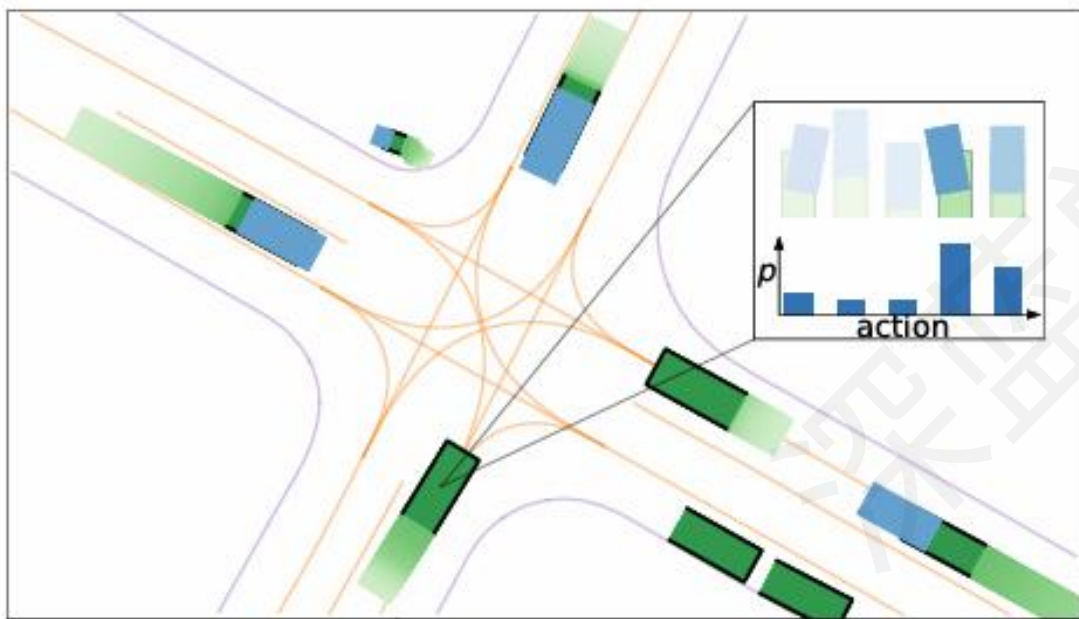
- Object-based: 可能损失场景信息，并且对未来物体不确定性的表征有局限
- Sensor-to-plan: 由于用专家信息训练，难以修正本身的错误，而且缺少可解释性
- Occupancy-based: 是可解释的object-free范式，但是需要集中大量资源

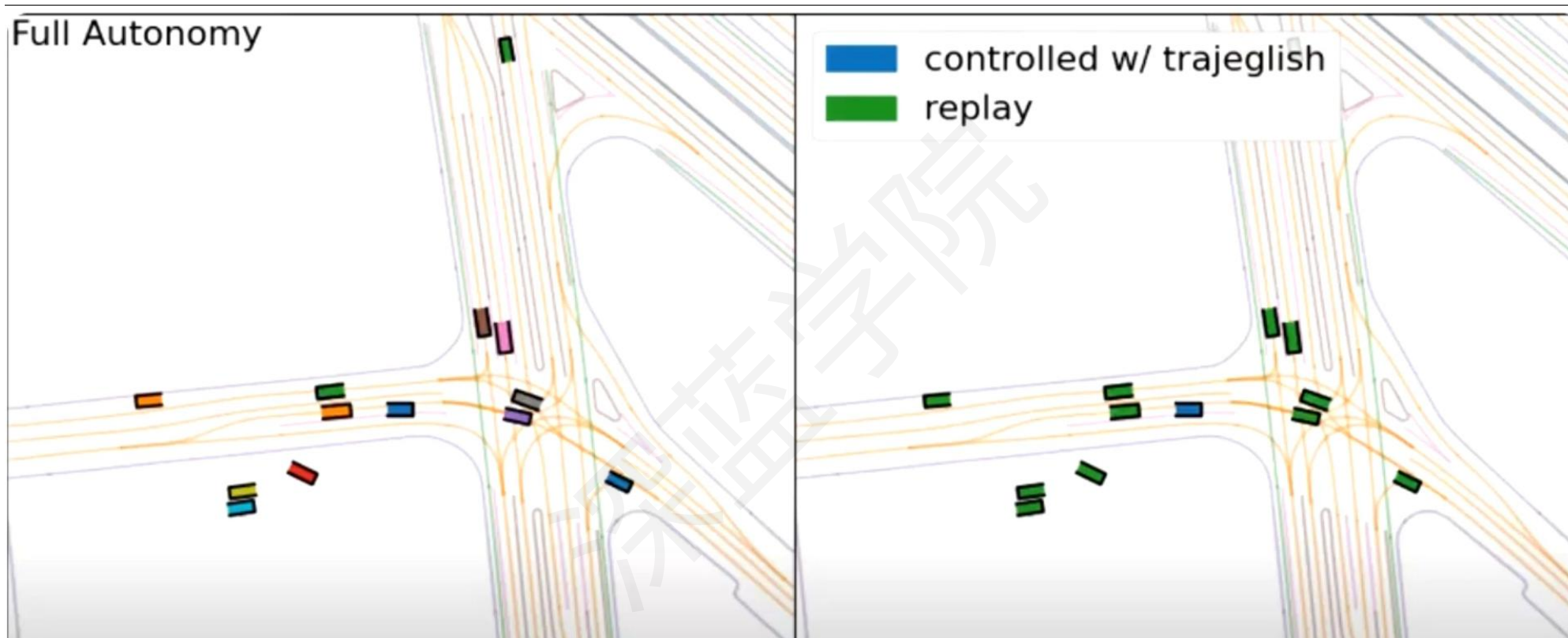




思想

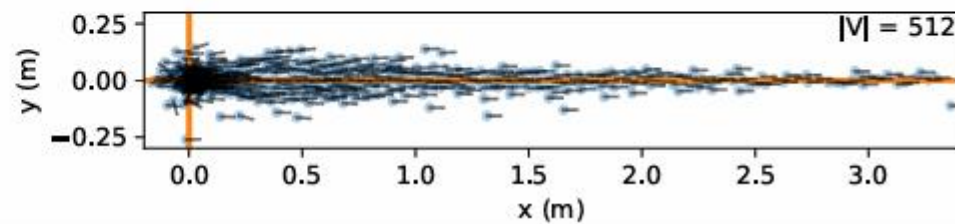
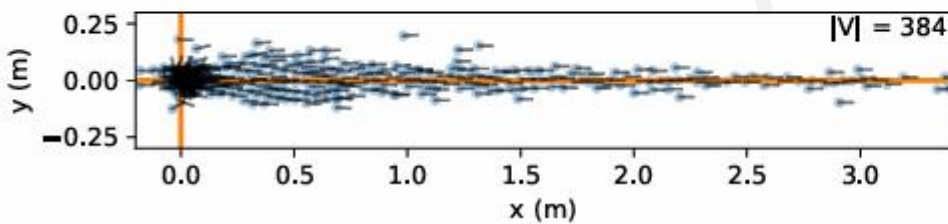
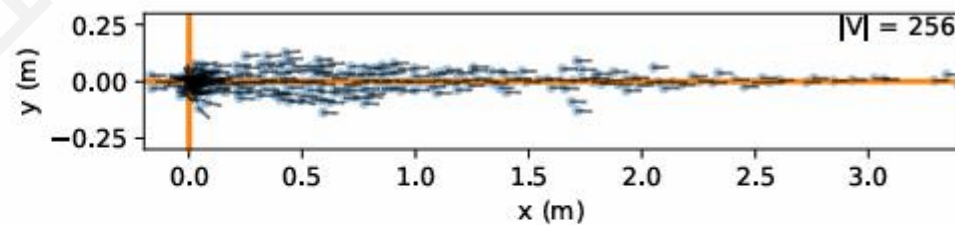
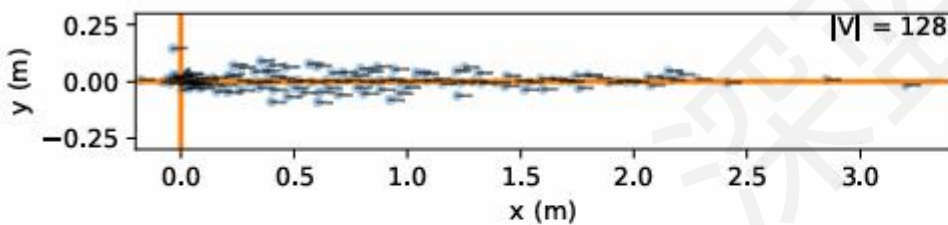
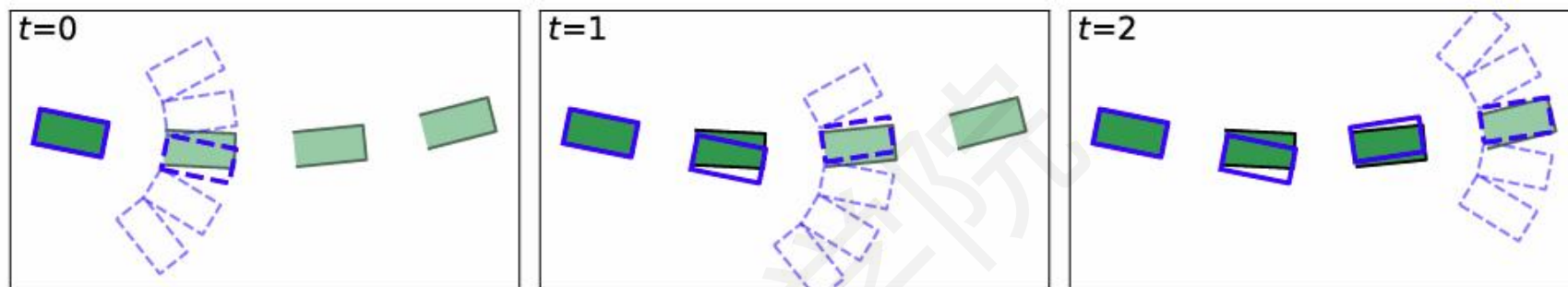
- 提出了一个道路智能体的自回归模型，已知地图信息、智能体历史状态和未来决策时，模型可以模拟出人类如何对此做出反应。
- 模型输出智能体在每个时间步可能状态的类别分布。





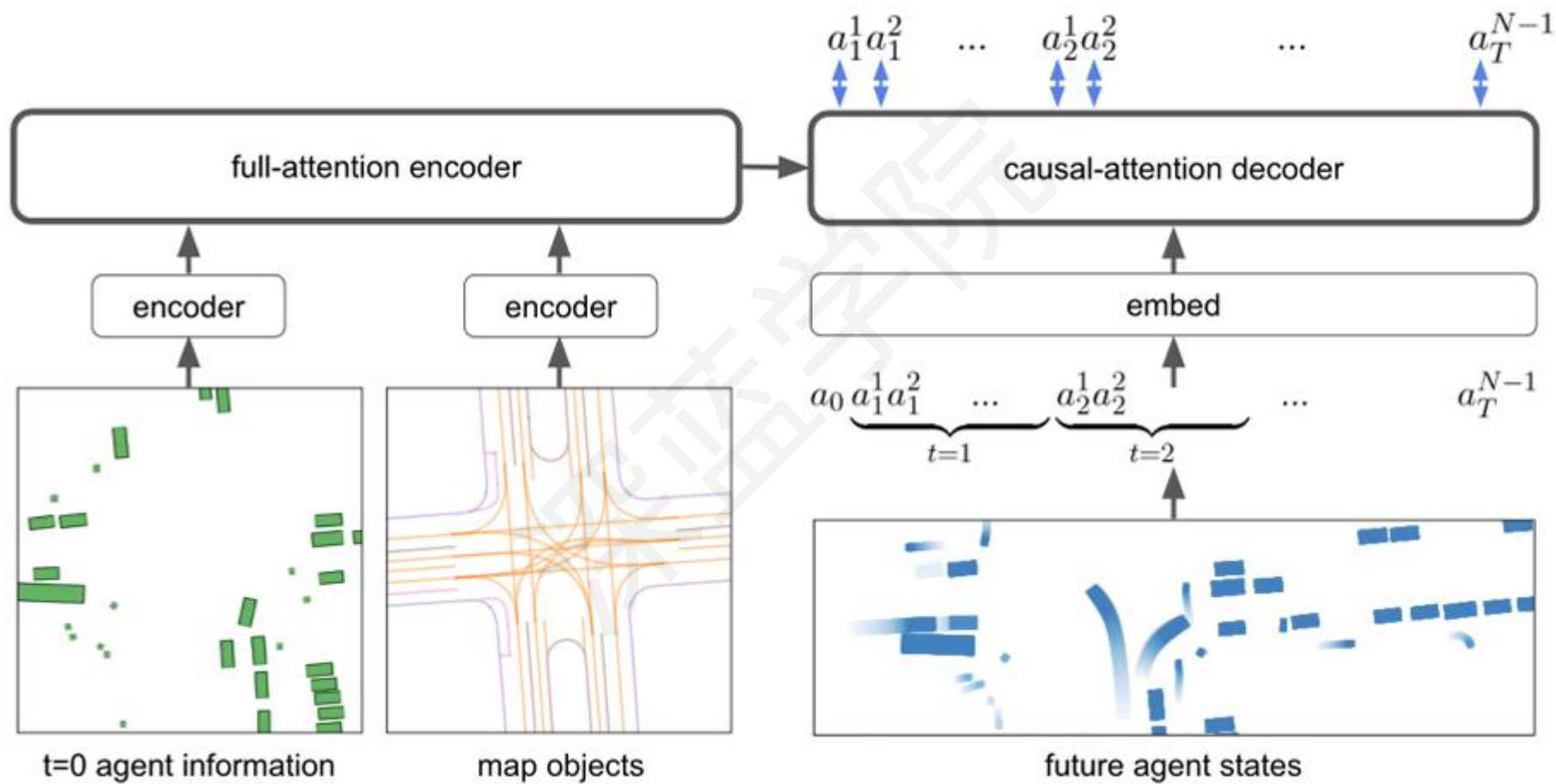


Token化





□ 模型





□ 特点

- 使用简单数据驱动的方法成功token化WOMD。
- 使用transformer-based方法建模motion-token序列，每刻输出一个智能体行为的分布。
- 成功提升了自驾模拟器的现实性，有助于增强自驾系统的安全性。

深蓝学院





大纲



生成模型

模仿学习

强化学习

深蓝学院





Learning Realistic Traffic Agents in Closed-loop



▣ 视频演示



CoRL

Learning Realistic Traffic Agents in Closed-loop

Chris Zhang, James Tu, Lunjun Zhang, Kelvin Wong, Simon Suo, Raquel Urtasun



UNIVERSITY OF
TORONTO

7th Conference on Robot Learning (2023), Atlanta, USA





Learning Realistic Traffic Agents in Closed-loop



□ 背景

- 现实的交通模拟对于安全且可扩展地开发自动驾驶软件至关重要。
- 模仿学习 (IL) 和强化学习 (RL) 的局限性
 - 模仿学习：直接从离线收集的真实世界观测数据中学习人类的交通行为，但没有明确的交通规则规范，不现实的违规行为，可能导致如碰撞和偏离道路的情况
 - 强化学习：可以训练智能体避免违规行为，但仅使用强化学习会导致不类人的驾驶行为
- 现有仿真方法的不足
 - 现有方法要么重放记录的轨迹而缺乏反应性，要么使用启发式策略，导致僵硬的行为；
 - 数据驱动的方法，如模仿学习，虽然提供了人类驾驶的监督信号，但缺乏明确的交通规则知识，容易产生不现实的策略；
 - 强化学习依靠手工设计的奖励来惩罚违规行为，无法捕捉人类驾驶的所有细微差别。





Learning Realistic Traffic Agents in Closed-loop



□ 主要贡献

- 提出 Reinforcing Traffic Rules (RTR) 方法

- RTR 是一种全面的闭环学习目标，通过交通合规约束下匹配专家示范数据，自然形成联合的 IL + RL 方法，结合了两者的优点；
- RTR 在现实数据和仿真生成的长尾场景的闭环模拟中学习。

- RTR 方法的优化

- 与先前结合 IL 和 RL 的方法不同，RTR 的闭环方法允许模型理解其行动的效果，显著减少累积误差；
- 利用模拟的长尾场景，通过让策略暴露于更有趣的互动中，提高学习效果，这在实际世界中大规模收集是困难且危险的。

- 实验结果

- RTR 学习到了更现实且更具泛化性的交通模拟策略，在训练中未见的常规和长尾场景中表现出色；
- RTR 生成的数据用于训练预测模型时，显著改进了下游预测指标，进一步证明 RTR 学习到了更现实的交通模拟。



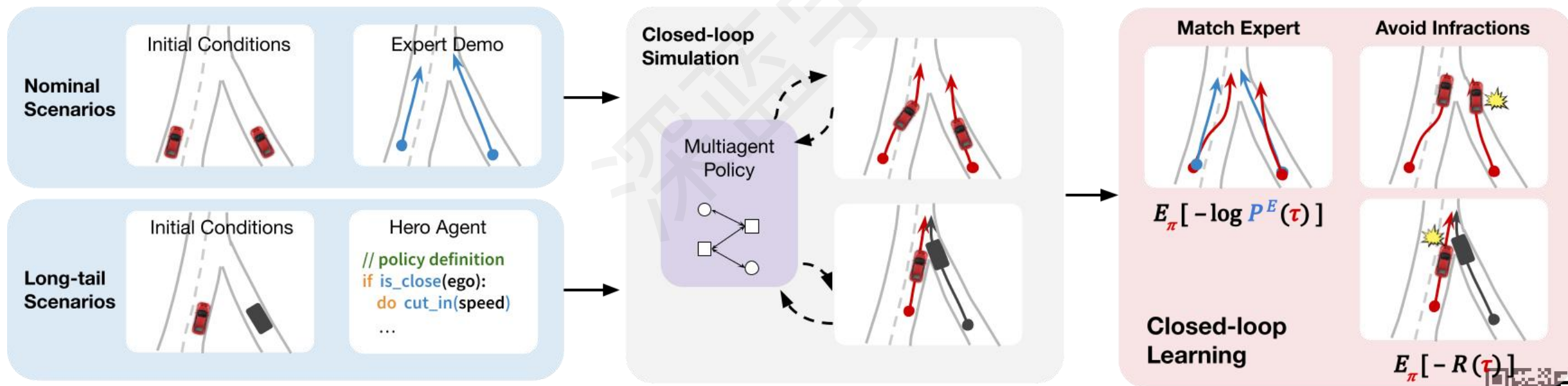


Learning Realistic Traffic Agents in Closed-loop



□ 整体流程

- 收集名义上的离线数据和模拟长尾场景。
- 统一学习目标：提出一个统一的学习目标，在违规约束下匹配专家示范数据，以学习到现实且无违规行为的智能体。
- 联合闭环 IL + RL 方法：构建一个联合的闭环 IL + RL 方法，既可以从离线收集的真实人类驾驶数据中学习，也可以从额外的模拟长尾场景中学习。

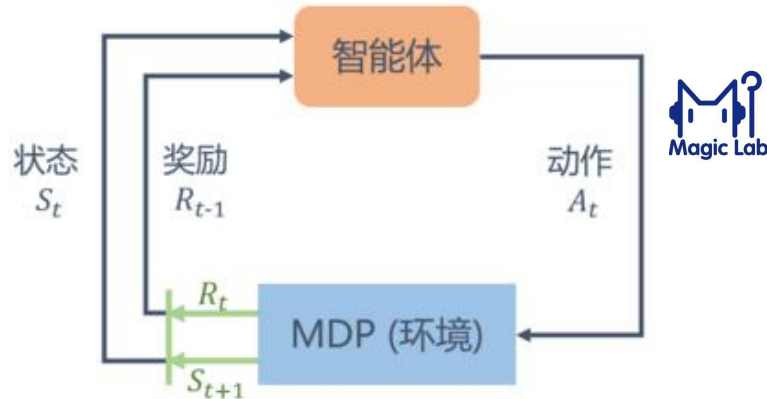




Learning Realistic Traffic Agents in Closed-loop

问题表述

- 将多智能体交通模拟建模为一个马尔可夫决策过程 (MDP)
- 状态、动作和策略
 - 状态 $s = \{s(1), \dots, s(N), m\} \in S$: 包括 N 个智能体的联合状态, 以及捕捉道路与车道拓扑的高清地图 m 。第 i 个智能体的状态 $s(i)$ 由其位置、航向和过去 H 个时间步的速度参数化;
 - 动作 $a = \{a(1), \dots, a(N)\} \in A$: 包括所有智能体的联合动作。第 i 个智能体的动作 $a(i)$ 由其加速度和转向角参数化;
 - 智能体由一个集中式策略 $\pi(a | s)$ 控制, 该策略将联合状态映射到联合动作。
- 轨迹和动态
 - 轨迹 $\tau_{0:T} = (s_0, a_0, \dots, s_{T-1}, a_{T-1}, s_T)$: 定义为所有智能体的状态-动作转换序列, 长度为 T ;
 - 使用运动学自行车模型作为转移动态模型 $P(s_{t+1} | s_t, a_t)$ 。
- 奖励
 - 每个智能体的奖励 $R(i)(s, a(i))$ 取决于所有智能体的状态, 以建模诸如碰撞等交互。联合奖励 $R(s, a) = \sum_i R(i)(s, a(i))$
- 策略学习
 - 模仿学习 (IL): 可以描述为 f -散度最小化问题: $\pi^* = \arg\min_{\pi} D_f(P^\pi(\tau) \| P^E(\tau))$, 其中 P^E 是专家诱导的分布;
 - 强化学习 (RL): 旨在找到最大化预期奖励的策略: $\pi^* = \arg\max_{\pi} E_{P^\pi}[R(\tau)]$ 。





Learning Realistic Traffic Agents in Closed-loop

□ 算法设计

- 通过反向KL散度最小化问题来匹配专家分布，并在违规约束下进行优化

$$\arg \min_{\pi} D_{\text{KL}}(P^{\pi}(\tau) \| P^E(\tau)) \quad R^{(i)}(\mathbf{s}, a^{(i)}) = \begin{cases} -1 & \text{if infraction} \\ 0 & \text{otherwise} \end{cases}$$

$$\text{s.t. } \mathbb{E}_{P^{\pi}}[R(\tau)] \geq 0$$

- 使用真实世界驾驶数据集 D 作为名义专家轨迹 $\tau^E \sim P^E$ ；以及额外模拟长尾场景，这些场景包含“英雄”智能体，诱导有趣的互动

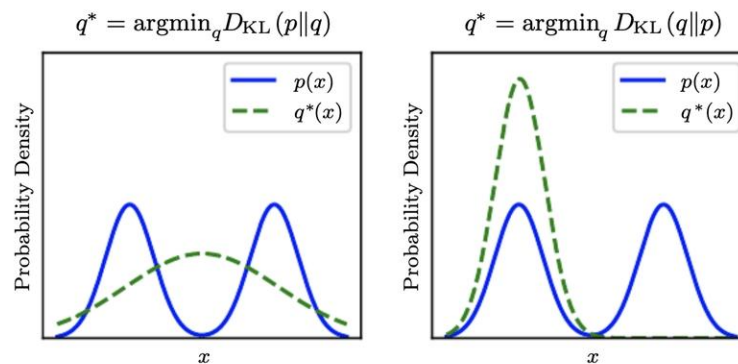
$$\pi(\mathbf{a}_{i,t} | \mathbf{s}_t) = \begin{cases} \pi_{s_0}^S(a_t^{(i)} | \mathbf{s}_t) & \text{if agent } i \text{ is hero} \\ \pi_{\theta}(a_t^{(i)} | \mathbf{s}_t) & \text{otherwise.} \end{cases}$$

- 目标函数分解为IL和RL组件。 L_{IL} 和 L_{RL} 联合优化，通过策略分布 $P^{\pi}(\tau)$ 进行期望计算

$$L = \mathbb{E}_{P^{\pi}} \left[\underbrace{-\log P^E(\tau)}_{\text{IL}} - \underbrace{\lambda R(\tau)}_{\text{RL}} \right] - H(\pi) = L^{\text{IL}} + \lambda L^{\text{RL}} - H(\pi)$$

- 模仿学习组件 L_{IL} 仅在专家示范时定义，并仅应用于名义数据。损失是策略引发轨迹与真实轨迹之间的距离

$$L^{\text{IL}} = \mathbb{E}_{\tau^E \sim D} \left[\mathbb{E}_{\tau \sim P^{\pi}(\cdot | \mathbf{s}_0^E)} [D(\tau^E, \tau)] \right]$$

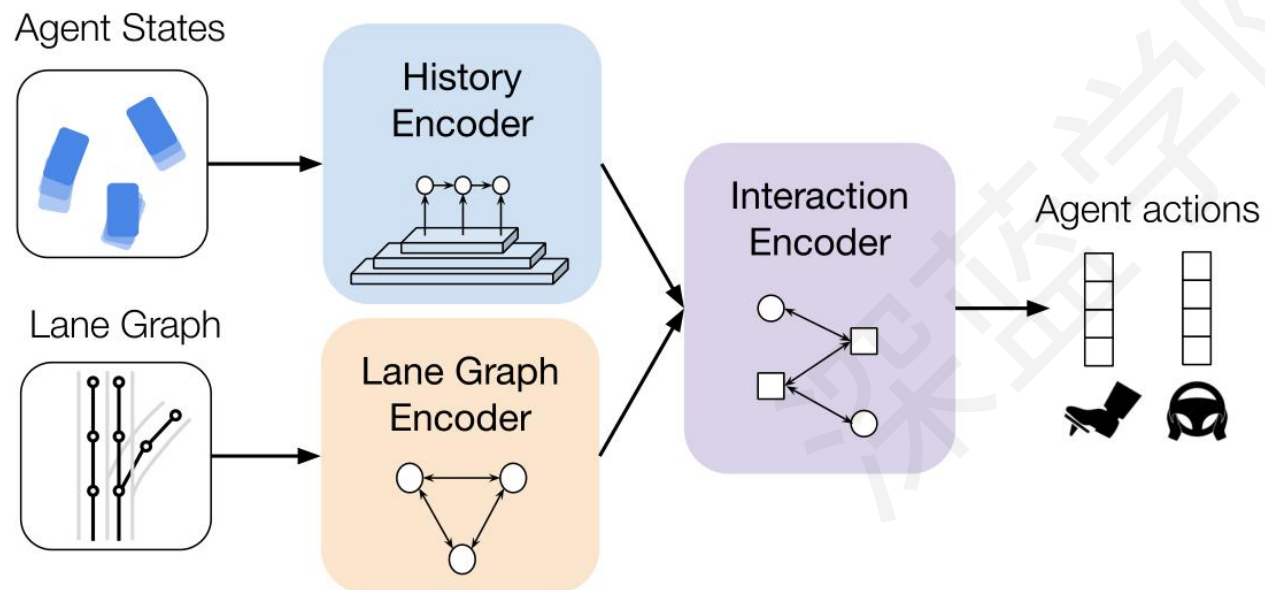




Learning Realistic Traffic Agents in Closed-loop

网络结构

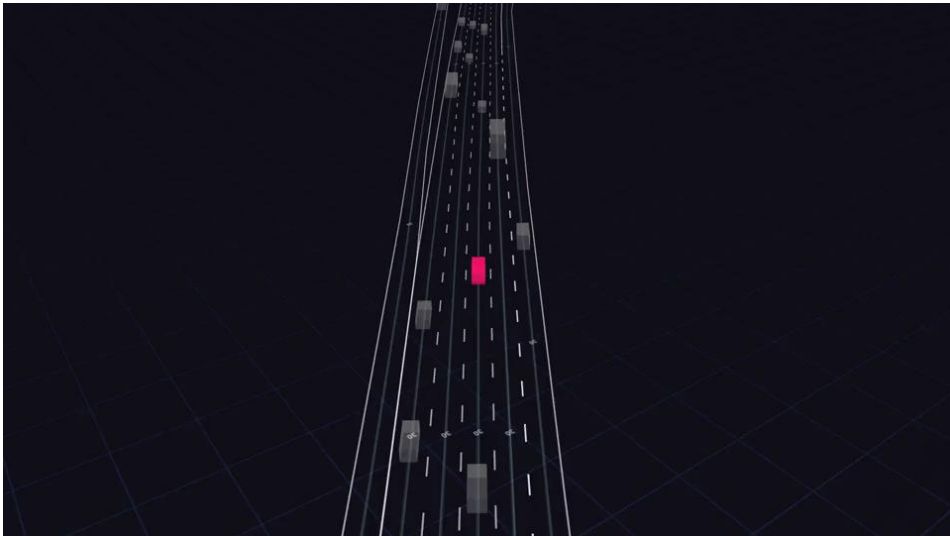
- 策略模型 π 架构使用了SOTA交通智能体运动预测文献中的常用思想，以提取上下文信息和地图特征并预测智能体动作；
- 使用共享的1D CNN和GRU来提取交通参与者历史上下文特征；
- 使用GNN提取地图特征。



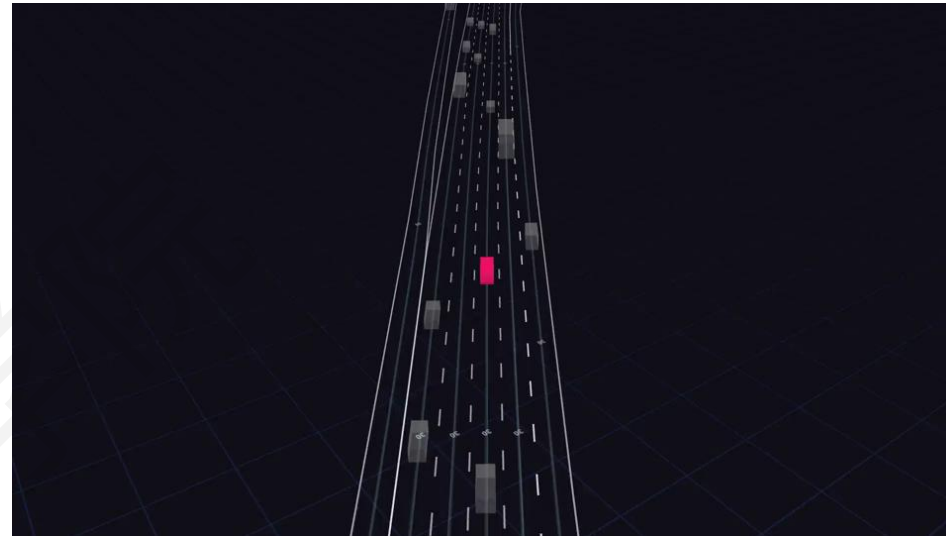


Learning Realistic Traffic Agents in Closed-loop

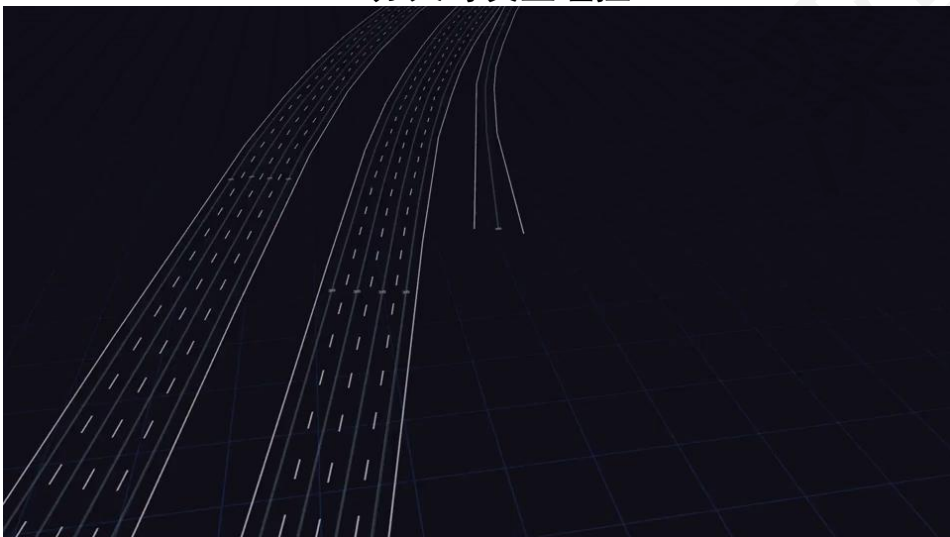
□ 定性比较（名义场景：左侧为IL，右侧为RTR）



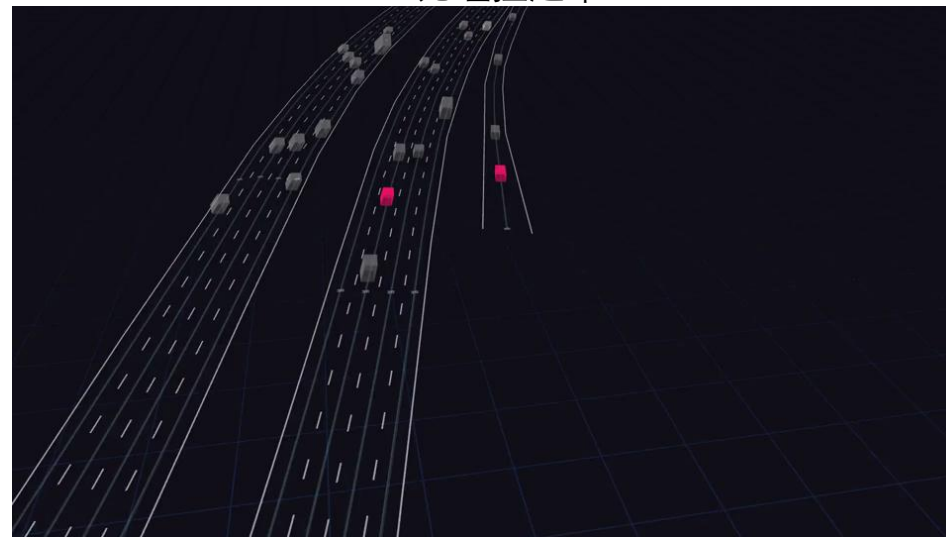
分叉时发生碰撞



无碰撞超车



合并时发生碰撞



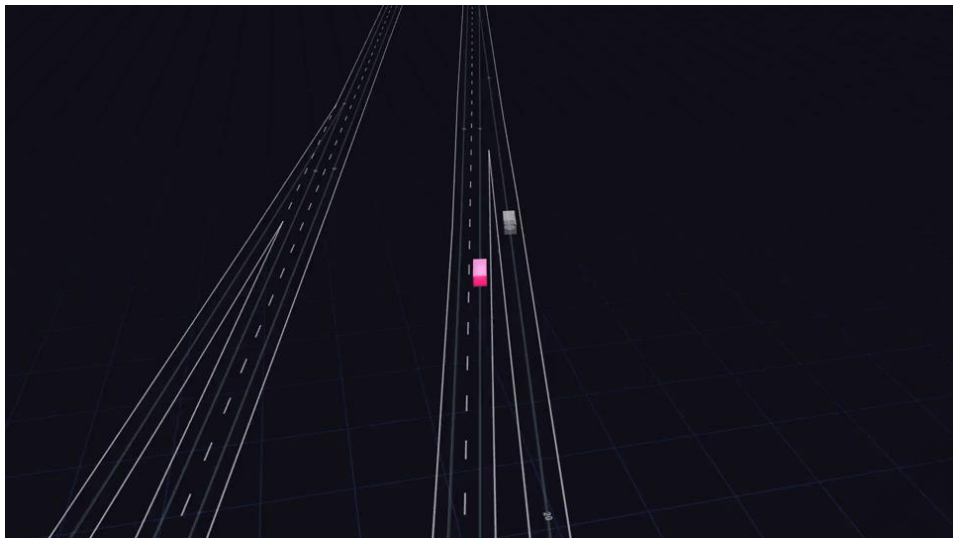
合并期间更安全的驾驶



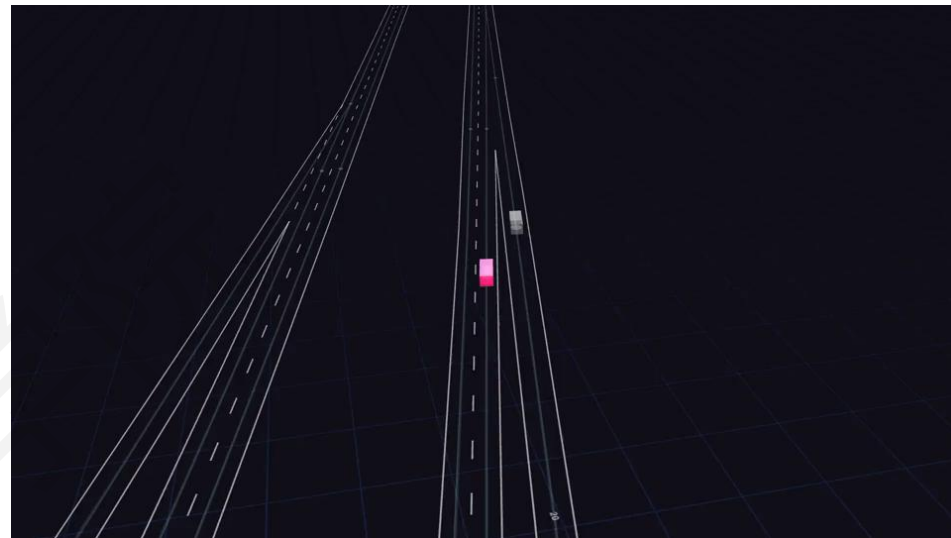


Learning Realistic Traffic Agents in Closed-loop

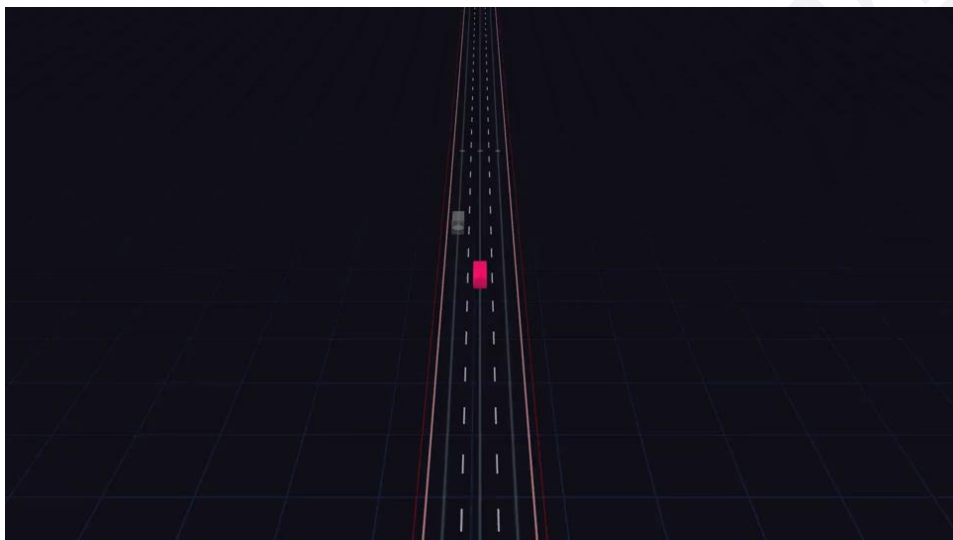
□ 定性比较（长尾场景）



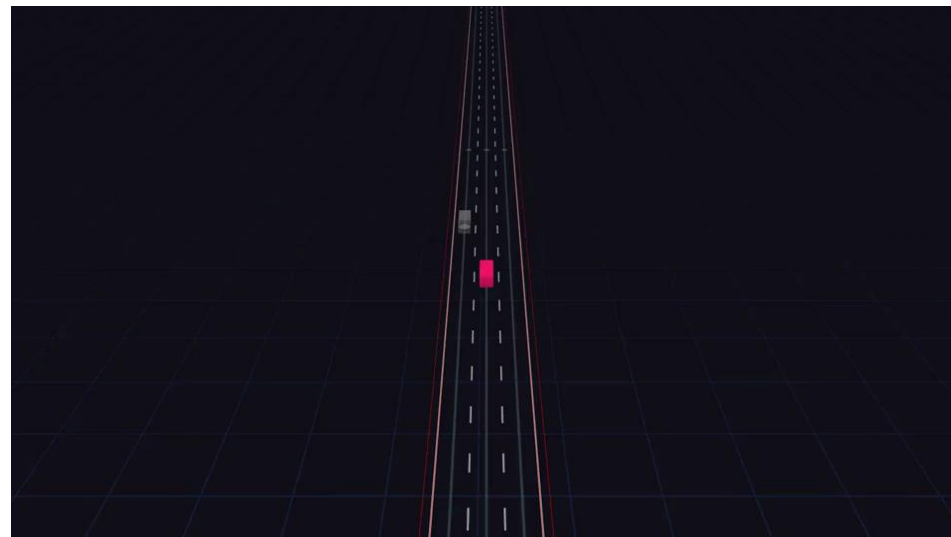
对合并参与者无反应



屈服于合并参与者



对插入者无反应



合并期间更安全的驾驶

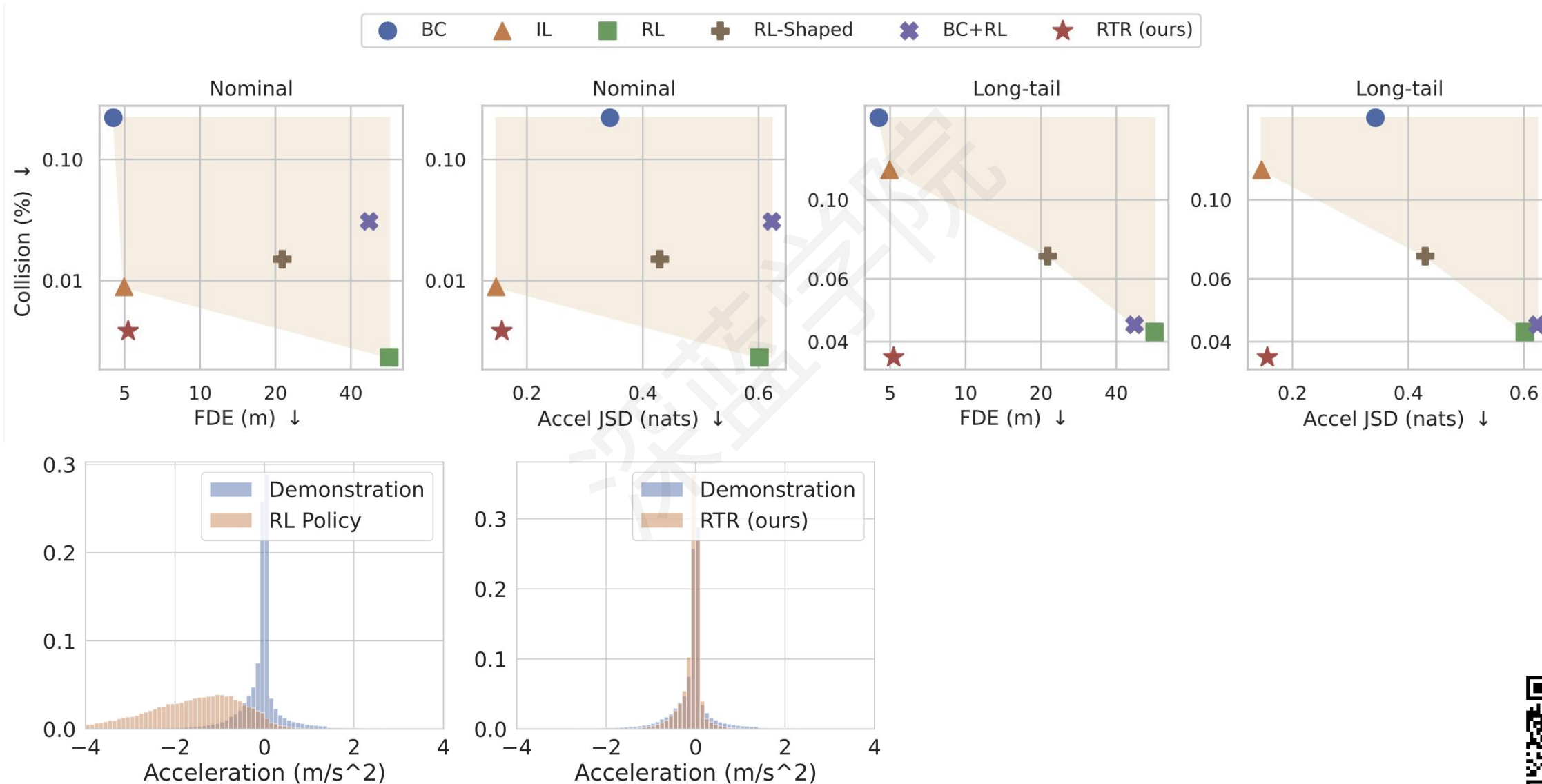




Learning Realistic Traffic Agents in Closed-loop



定量比较（真实性以及避免违规）

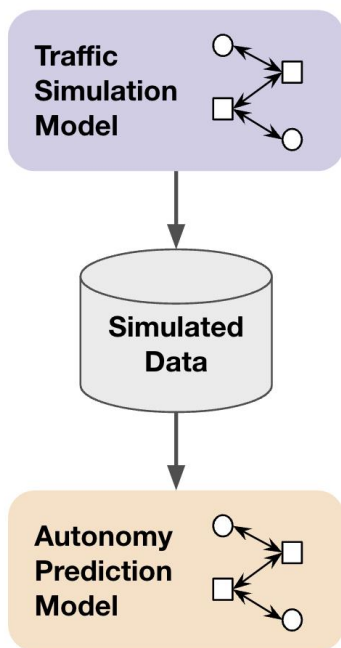




Learning Realistic Traffic Agents in Closed-loop

▣ 下游评估

- 在 RTR 模拟数据上训练的预测模型在保留的真实数据上实现了最佳指标，这表明 RTR 模拟更真实



Method	FDE (m)	CTE (m)
BC	2.44 ± 0.05	0.90 ± 0.04
IL	1.75 ± 0.06	0.28 ± 0.01
RL	15.42 ± 1.21	0.32 ± 0.02
RL-Shp	6.66 ± 0.26	0.33 ± 0.01
BC+RL	9.06 ± 0.50	0.42 ± 0.03
RTR	1.58 ± 0.05	0.27 ± 0.03





Think2Drive: Efficient Reinforcement Learning by Thinking in Latent World Model for Quasi-Realistic Autonomous Driving



□ 背景

- 自动驾驶挑战
 - 真实世界的自动驾驶（AD）涉及许多极端情况。最新发布的CARLA v2 AD模拟器增加了驾驶场景事件。现有的工作主要依赖于特定的规则进行规划，但这些规则无法覆盖CARLA v2中更复杂的情况。
- 模拟器的改进
 - CARLA v2相比于CARLA v1提供了更接近现实的测试平台，但目前尚无文献报告在CARLA v2的新场景中取得成功。现有方法主要依赖特定规则，难以灵活有效地处理复杂情况。
- 数据效率和灵活性
 - 现有的自动驾驶RL方法大多是无模型的，这在处理CARLA v2的复杂性时会面临数据效率低的问题。通过模型驱动的RL方法，可以提高数据效率并灵活应对复杂场景。





Think2Drive: Efficient Reinforcement Learning by Thinking in Latent World Model for Quasi-Realistic Autonomous Driving



□ 主要贡献

- 开发了首个针对自动驾驶的基于模型的强化学习方法——Think2Drive
 - 这个方法通过一个世界模型来学习环境的转换，然后作为一个神经模拟器来训练规划器。这显著提高了训练效率，因为世界模型可以在低维状态空间中进行并行计算。
- 提出的CornerCaseRepository基准
 - 提出了一个支持按场景评估驾驶模型的基准，并引入了一个新的平衡度量，结合了路线完成率、违规次数和场景密度，以更全面地评估驾驶性能。
- 实现专家级表现
 - 在CARLA v2中达到专家级表现，这是迄今为止在CARLA v2上首次实现100%路线完成率的报告。
- 三大技术障碍及解决方案





Think2Drive: Efficient Reinforcement Learning by Thinking in Latent World Model for Quasi-Realistic Autonomous Driving



□ 问题表述

Think2Drive利用特权信息进行规划，并通过世界模型实现有效的训练，而无需直接与物理模拟器进行频繁的交互。

- 特权信息输入 x_t ：周围车辆和障碍物的边界框、高清地图、交通灯状态等，特权信息在规划过程中直接使用的高质量、高精度的环境信息，消除感知带来的影响；
- 控制信号输出 A ：油门、转向和刹车；
- 规划器和世界模型
 - 规划器模型 (π_η)：基于当前状态 s_t ，输出行动 a_t ；
 - 世界模型 (F)：学习驾驶场景的转换，使规划器可以通过“思考”而不是直接与物理模拟器交互进行驾驶和训练。
- “思考”迭代过程
 - 在时间步 t ，给定初始输入 x_t ，世界模型将其编码为状态 s_t ；
 - 规划器基于 s_t 生成行动 a_t ；
 - 世界模型根据 s_t 和 a_t 预测奖励 r_t 、终止状态 c_t 和未来状态 s_{t+1} 。
- 整体流程
$$s_t \leftarrow F_\theta^{Enc}(x_t), \quad a_t \leftarrow \pi(s_t), \quad s_{t+1} \leftarrow F_\theta^{Pre}(s_t, a_t)$$





Think2Drive: Efficient Reinforcement Learning by Thinking in Latent World Model for Quasi-Realistic Autonomous Driving



□ 世界模型学习

● 世界模型组件

- RSSM (Recurrent State Space Model) : 用于在潜在空间中提供环境的准确转换函数, 将状态表示分解为随机表示 z_t 和确定性隐藏状态 h_t , 以更好地建模真实转换函数中的确定性和随机性方面,

$$\text{RSSM} \left\{ \begin{array}{ll} \text{Sequence model:} & h_t = f_{\theta}(h_{t-1}, z_{t-1}, a_{t-1}) \\ \text{Encoder:} & z_t \sim q_{\theta}(z_t | h_t, x_t) \\ \text{Dynamics predictor:} & \hat{z}_t \sim p_{\theta}(\hat{z}_t | h_t) \\ \text{Reward predictor:} & \hat{r}_t \sim p_{\theta}(\hat{r}_t | h_t, z_t) \\ \text{Termination predictor:} & \hat{c}_t \sim p_{\theta}(\hat{c}_t | h_t, z_t) \\ \text{Decoder:} & \hat{x}_t \sim p_{\theta}(\hat{x}_t | h_t, z_t) \end{array} \right.$$

● 训练损失

- 预测损失 (L_{pred})
- 动态损失 (L_{dyn})
- 表示损失 (L_{rep})

$$\mathcal{L}_{pred}(\theta) \doteq -\ln p_{\theta}(x_t | z_t, h_t) - \ln p_{\theta}(r_t | z_t, h_t) - \ln p_{\theta}(c_t | z_t, h_t)$$

$$\mathcal{L}_{dyn}(\theta) \doteq \max(1, \text{KL}[fz(q_{\theta}(z_t | h_t, x_t)) || p_{\theta}(z_t | h_t)])$$

$$\mathcal{L}_{rep}(\theta) \doteq \max(1, \text{KL}[q_{\theta}(z_t | h_t, x_t) || fz(p_{\theta}(z_t | h_t))])$$

$$\mathcal{L}(\theta) \doteq \mathbb{E}_{q_{\theta}} \sum_{t=1}^T (\beta_{pred} \mathcal{L}_{pred}^t(\theta) + \beta_{dyn} \mathcal{L}_{dyn}^t(\theta) + \beta_{rep} \mathcal{L}_{rep}^t(\theta))$$





Think2Drive: Efficient Reinforcement Learning by Thinking in Latent World Model for Quasi-Realistic Autonomous Driving



□ 规划器学习

- Actor-Critic架构
 - Actor (规划器模型) : 规划器模型负责生成行动;
 - Critic (评价模型) : 评价模型用于辅助规划器模型的学习。
- 训练步骤
 - 初始输入: 从记录中取出输入 x_t 作为起点, 世界模型将其映射为 $s_t = (z_t, h_t)$;
 - 探索过程: 世界模型和规划器模型进行 T 步探索, 生成序列 $\langle \hat{s}_{1:T}, a_{0:T}, r_{0:T}, c_{0:T} \rangle$;
 - 目标: 规划器模型 π_η 尝试最大化预期折扣回报;
 - 评价: 评价模型学习根据规划器的策略评估每个状态 $V(s_T) \approx E_{s \sim F_\theta, a \sim \pi_\eta}(R_t)$ 。
- 训练损失
 - 评价模型损失: 使用交叉熵损失训练评价模型
 - 规划器模型损失: 用于训练actor的奖励预期项通过移动统计进行归一化

$$\mathcal{L}_{\text{critic}}(\psi) \doteq - \sum_{t=1}^T y_t^\top \ln p_\psi(\cdot | s_t)$$

$$\mathcal{L}(\theta) \doteq \sum_{t=1}^T \left(E_{\pi_\eta, p_\theta} \left[\frac{\text{fz}(R_t^\lambda)}{\max(1, S)} \right] - \beta_{en} H[\pi_\eta(a_t | s_t)] \right)$$





Think2Drive: Efficient Reinforcement Learning by Thinking in Latent World Model for Quasi-Realistic Autonomous Driving

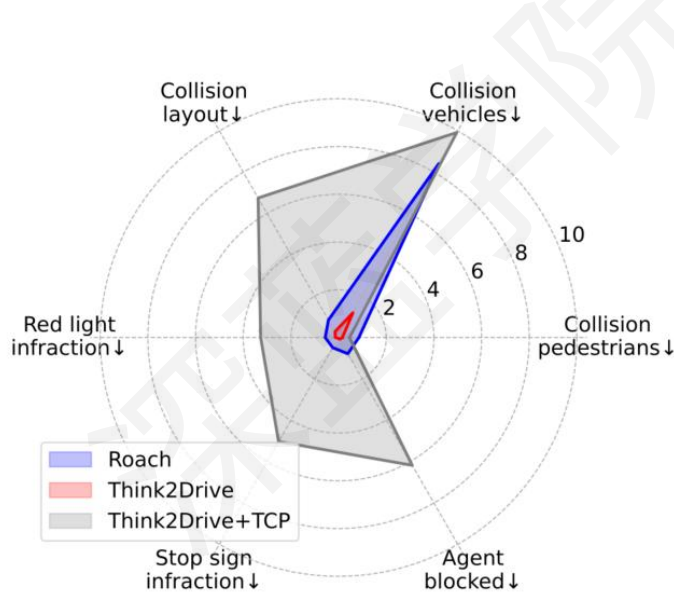


CornerCaseRepository 基准测试集

- 解决了官方基准测试中因单一路线包含多个场景导致的训练和评估困难问题
- 提供更精细化的场景评估，以便于调试和改进模型



(a) Driving performance.



(b) Infractions per kilometer.





总结



□ 数据驱动的规划方法

- 生成模型

优点：高逼真度和复杂性，适应性强

缺点：计算密集，难以处理稀有行为

- 模仿学习IL

优点：类人行为，数据利用率高，实现简单

缺点：规则遵循不足，泛化问题

- 强化学习RL

优点：规则遵循，探索性学习

缺点：类人行为不足，样本效率低

- 数据驱动方法的未来趋势

生成稀有场景，具备长尾案例下的泛化能力

提高数据利用效率

在遵循交通规则的同时保持类人行为

